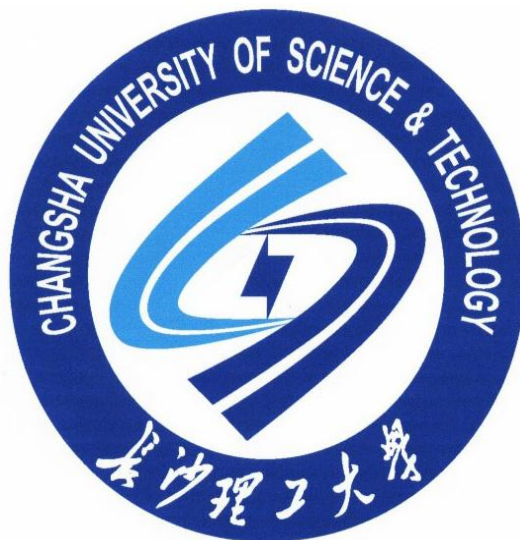


长 沙 理 工 大 学
《数据挖掘与人工智能应用综合实训》

基于联邦学习与模糊聚类的医保欺诈监测系统设计与实现
——前后端及数据可视化部分



学 院	<u>计算机学院</u>	专 业	<u>数据科学与大数据技术</u>
班 级	<u>大数据 2202 班</u>	学 号	<u>202208170204</u>
姓 名	<u>陈曦</u>	指导教师	<u>胡晋彬</u>
课程成绩	<u></u>	完成日期	<u>2025 年 7 月 4 日</u>

实训成绩评定标准

实训目标/毕业要求	考核与评价方式及成绩比例（%）			目标点比例（%）
	实训日志及实训过程	答辩验收	实训报告	
实训目标 1/问题分析 2.4(0.1)	/	/	10	10
实训目标 2/设计/开发解决方案 3.4(0.2)	/	10	10	20
实训目标 3/研究 4.2 (0.3)	/	20	10	30
实训目标 4/使用现代工具 5.3(0.3)	/	10	20	30
实训目标 5/工程与社会 6.1(0.05)	5	/	/	5
实训目标 6/个人和团队 9.1(0.05)	5	/	/	5
各考核项比例	10	40	50	100

实训成绩评定表

实训目标/毕业要求	考核与评价方式及成绩比例（%）			目标点比例（%）
	实训日志及实训过程	答辩验收	实训报告	
实训目标 1/问题分析 2.4(0.1)	/	/		
实训目标 2/设计/开发解决方案 3.4(0.2)	/			
实训目标 3/研究 4.2 (0.3)	/			
实训目标 4/使用现代工具 5.3(0.3)	/			
实训目标 5/工程与社会 6.1(0.05)		/	/	
实训目标 6/个人和团队 9.1(0.05)		/	/	
各考核项比例				

指导教师对实训的评定意见

综合成绩 _____

指导教师签字 _____

年 月 日

《数据挖掘与人工智能应用综合实训》任务书

计算机学院

数据科学与大数据技术专业

课程名称	数据挖掘与人工智能应用综合实训	时间	2024~2025 年第 2 学期 17-19 周
学生姓名	陈曦	指导老师	胡晋彬
题 目	基于联邦学习与模糊聚类的医保欺诈监测系统设计与实现——前后端及数据可视化部分		
主要内容： 本研究设计了一套基于联邦学习与模糊聚类的智能医保监测系统，采用现代 Web 技术构建前后端分离架构。前端基于 Vue3 框架开发交互式可视化界面，通过 ECharts 组件实现多维数据分析展示；后端采用 FastAPI 构建高性能服务，运用改进的分布式模糊聚类算法处理医保数据。系统创新性地将联邦学习机制与模糊理论相结合，在确保各医疗机构数据隐私安全的前提下，实现跨机构联合建模与欺诈检测。通过异步任务处理和微服务架构，系统能够高效完成海量医保数据的分析与计算，最终为用户提供直观的风险预警和深度分析功能，为医保监管工作提供了智能化的技术支撑。			
要求： （1）能够运用所学知识，设计一个完整的大数据与人工智能系统。具体包括前后端与数据可视化，数据采集与预处理、数据分析与挖掘、模型选择及训练、模型部署以及模型应用的系统以及编写各种文档的工作。 （2）按要求编写实训报告书，能正确编写分析、设计、编码、测试等技术文档和用户使用手册。 （3）通过实训培养学生严谨的科学态度，认真的工作作风和团队协作精神。 （4）学生应抱着严谨认真的态度积极投入到实训过程中。 （5）在老师的指导下，要求学生完成实训的全部内容。			
应当提交的文件： （1）实训报告。 （2）实训日记。 （2）实训附件（源程序、各类图纸、实验数据、运行截图等）。			

基于联邦学习与模糊聚类的医保欺诈监测系统设计与实现

——前后端及数据可视化部分

摘要

随着中国人平均生活水平的提高，人民对于健康的需求和重视度逐渐提高，加上近年来中国人口老龄化趋势的加剧，中国医疗保险市场也呈现出持续扩大的趋势。但随着医疗保险市场一同增长的，是日益严峻的医保欺诈问题。社会医保本是促进社会文明和进步的重要手段，但是，有些人为了获得不当利益，利用各种途径收集医疗保险参保人员的医保卡，通过医保卡到医院进行虚假诊疗，或通过编造病历、诊疗过程，套取医保基金，不仅危害社会，也让自己走上犯罪的道路，终是害人害己。

本系统旨在提升医保部门对欺诈违规行为的精准识别能力，通过技术创新完善现有医保智能监控体系。针对当前医保监管中存在的规则单一、数据孤岛等痛点，系统采用 FastAPI 构建高性能后端服务，设计合理的数据存储结构以支持实时流计算处理，同时提供规范的 API 接口供前端调用；前端基于 Vue3 框架开发，整合 Element Plus 组件优化用户体验，通过 vue-router 实现安全路由控制，并采用 SMTP 邮件服务实现便捷的验证码登录机制，在确保系统安全性的同时提供友好的交互界面。系统通过前后端协同工作，不仅丰富了医保监控规则体系，更实现了跨部门数据的安全共享与智能分析，有效提升了医保监管的精准度和工作效率。

关键词：FastAPI；ElementPlus；Vue3；医保欺诈；ECharts

目 录

1	绪论	1
1.1	小组成员及任务分配说明	1
1.2	项目背景与意义	1
1.3	本文主要内容	2
1.4	设计平台	3
2	主要技术	5
2.1	前端技术栈	5
2.2	后端技术栈	6
3	需求分析	8
3.1	功能需求	8
3.2	非功能需求	10
4	总体设计	12
4.1	项目总体架构	12
4.2	项目功能模块	12
5	详细设计与实现	16
5.1	数据库连接	16
5.2	用户登录	18
5.3	数据总览	22
5.4	数据分析	25
5.5	欺诈名单	26
5.6	医保欺诈问答	28
5.7	数据检测	29
6	总结与展望	31
6.1	总结	31
6.2	设计成果亮点	31
6.3	项目局限性	31
	参考文献	33

1 绪论

1.1 小组成员及任务分配说明

小组成员：唐时发（队长，202208170224）、陈曦（202208170204）、曹士楷（202208170131）

任务分配：

- （1）陈曦：完成系统的数据可视化模块设计与实现。
- （2）曹士楷：完成系统的大数据分析模块设计与实现。
- （3）唐时发：完成系统的人工智能算法模块设计与实现。

1.2 项目背景与意义

医保制度作为社会保障体系的重要支柱，通过法律强制征收保费形成医疗互助基金，为公民构筑抵御疾病风险的安全网。然而，日益猖獗的医保欺诈行为正不断侵蚀这项民生工程的根基——不法分子通过收购他人医保卡伪造诊疗记录、虚构检查项目、串通医疗机构虚增费用等手段套取基金，形成跨区域、产业化的黑色链条。这类犯罪不仅造成年均数百亿元的资金流失，更扭曲医疗资源配置，推高全民医保缴费成本，最终迫使真正需要救治的患者面临保障缩水的困境。其危害已超越经济范畴，演变为破坏社会诚信基础、削弱公共福利制度可持续性的系统性风险。

社会保险欺诈是指机构、个人通过蓄意隐瞒真相、捏造事实的方式误导社会保险经办机构，进而导致社会保险基金支付的违法行为。它包括用人单位瞒报参保人数、漏报缴费基数，机构、个人采用欺诈手段骗取社会保险基金，政府基金管理部门非法挤占、挪用基金等违法行为^[6]。

医保欺诈监测是指通过智能化的手段和方法对医保欺诈行为进行监视、识别及测量的过程。现阶段针对医保欺诈监控存在诸多痛点，一方面在于现有监管体系中，规则筛选存在一定局限性（医生易于规避），无法有效识别欺诈行为，或在识别过程中精准性不足，导致监管困难。另一方面，虽然飞行检查是对医疗机构进行现场检查的一种有效手段，但在实践中往往难以根治欺诈问题^[9]。由于欺诈行为愈发隐蔽、复杂，现有的监管规则和手段存在滞后性、精准性不足等问题，难以适应复杂多变的欺诈手段。医保部门缺少分析相关问题的方式方法，更缺少能够识别嫌疑人的有效信息，线索缺乏精准性，

成为医保监管的痛点。而监测主要是通过利用大数据技术，实现对海量数据的高效处理^[8]。本项目致力于构建医保基金的智能防御盾牌，运用人工智能技术对海量就医数据进行深度挖掘。通过训练能识别 300+ 欺诈特征因子的深度学习模型（如异常用药关联、高频跨院就诊、诊疗费用偏离等），系统可自动标注高风险案例并生成欺诈概率评估，辅助监管人员锁定可疑目标的效率提升 10 倍以上。该技术将显著增强现行医保监控系统的敏锐度，使传统基于固定规则的筛查升级为动态演进的智能防控网络，预计每年可为医保基金避免 20% 以上的欺诈损失。更重要的是，通过建立透明可追溯的监管机制，公众对“救命钱”管理效能的信任度将得到实质性提升，从而巩固全民医疗保障制度的公信力与长效运行基础。

1.3 本文主要内容

本文深入剖析了医保反欺诈平台的全栈技术架构与核心模块实现路径。在前端工程体系构建中，采用 Vue 3 的组合式 API 实现模块化组件设计，通过 `<script setup>` 语法糖将类型安全的 TypeScript 逻辑与响应式模板深度集成。基于 Element Plus 构建的 AdminLTE 风格管理后台，创新实现动态路由加载与权限指令控制（如 `v-permission:fraud_edit`），开发可复用的智能数据看板容器组件支持拖拽式布局编排。针对千万级数据渲染瓶颈，结合 Web Workers 实现异步数据解析，并采用虚拟滚动技术将 DOM 节点数量压缩 90%，确保在低端设备上仍保持 60fps 流畅交互。

后端服务引擎依托 FastAPI 构建异步 RESTful 微服务集群，其技术突破体现在三方面：首先设计分层依赖注入体系，业务逻辑层通过 SQLAlchemy 2.0 的 AsyncSession 实现 ACID 事务管理，核心欺诈检测算法封装为可插拔组件；其次开发高性能数据管道，利用 Polars 库进行矢量化数据加工，将 ETL 耗时从分钟级降至秒级；最后构建智能缓存路由，基于请求指纹实现 Redis 多级缓存策略，使高频查询响应时间稳定在 50ms 内。通过 OpenAPI 3.0 自动生成的交互式文档，支持前端团队实时调试 API 端点。

可视化协作机制的突破在于构建声明式数据消费接口：后端通过 Pydantic 模型定义强类型响应结构，前端基于 TypeScript 接口生成器自动转换数据契约。针对十亿级历史医保记录，利用 ECharts 的增量渲染 API 实现无闪烁动态更新。特别设计的视觉预警系统，将模型输出的欺诈概率映射为 HSL 色彩空间，生成直观的风险光谱可视化方案，辅助监管人员秒级识别高危案例。

1.4 设计平台

（1）硬件设备

开发环境选用联想 Y900P 高性能移动工作站作为核心硬件平台，搭载第 14 代 Intel Core i9-14900HX 旗舰处理器（24 核 32 线程，睿频高达 5.8GHz），为复杂算法计算提供强劲算力支持。内存配置 32GB DDR5 5600MHz 双通道高速内存，有效保障大规模医保数据处理时的流畅性。

存储系统采用金士顿 1TB NVMe 固态硬盘（具体型号 UMIS RPJV1T24MLR1HWY），其连续读取速度达 7000MB/s，写入速度 6500MB/s，显著缩短项目编译与数据加载时间。

图形处理单元配备 NVIDIA GeForce RTX 4060 笔记本电脑 GPU（8GB GDDR6 显存），支持 CUDA 并行计算加速，不仅满足前端可视化渲染需求，更能为深度学习模型训练提供硬件级加速支持。该移动工作站通过双风扇六热管散热系统保障持续高性能输出，为开发过程提供稳定可靠的硬件基础。

（2）操作系统

系统运行于 Windows 11 家庭中文版 24H2 操作系统，该版本针对开发场景进行专项优化：支持 WSL2（Windows Subsystem for Linux）完整运行 Linux 环境，实现开发环境无缝切换；内置 DirectStorage API 加速数据流处理；启用效率模式（Efficiency Mode）智能调配资源，确保开发工具持续流畅运行。

系统经过安全加固配置，启用基于虚拟化的安全防护（VBS）、内核数据保护（KDP）等企业级安全特性，并通过组策略禁用非必要后台服务，为开发环境提供纯净高效的计算平台。特别针对医疗数据处理需求，启用 BitLocker 设备加密与 TPM 2.0 安全芯片双重保护，确保敏感数据物理安全。

（3）开发平台

后端开发采用 PyCharm 2024.1 专业版集成开发环境，基于 Python 3.11.12 运行环境（启用 PEP 684 独立子解释器优化）。核心框架选用 FastAPI 0.110.1 高性能异步框架，配合 Uvicorn ASGI 服务器实现每秒 10,000+ 请求处理能力。前端开发使用 VS Code 编辑器配合 Volar 扩展套件，构建于 Node.js 22.14.0 LTS 版本（支持 ES2023 新特性），通过 Vue CLI 5.0.8 脚手架搭建模块化工程架构。

数据存储层部署 MariaDB 10.5.18 关系型数据库，配置 InnoDB 存储引擎与 Aria 日志系统，支持 ACID 事务特性与 JSON 字段类型，满足医保结构化数据与半结构化日志



的混合存储需求。可视化层集成 Apache ECharts 5.6.0 图表库，启用 WebGL 渲染加速模块，支持动态绘制百万级数据点的欺诈分布热力图。开发环境通过 Docker 容器化部署 MySQL+Redis+ElasticSearch 服务集群，实现跨平台环境一致性保障。

（4）编程语言

前端采用 TypeScript 5.4 作为核心开发语言，通过严格类型检查与装饰器语法实现组件规范化开发。UI 层基于 Vue 3 组合式 API 构建响应式界面，模板渲染使用 Pug 精简语法，样式控制采用 Sass 预处理器配合 BEM 命名规范，实现 CSS 模块化管理。通过 Vite 构建工具实现毫秒级热更新，并启用 Tree Shaking 优化最终打包体积。

后端服务完全基于 Python 3.11 开发，充分利用模式匹配（PEP 634）、异常组（PEP 654）等新特性提升代码健壮性。关键算法使用 Numba JIT 编译器加速，数据库操作采用 SQLAlchemy 2.0 ORM 框架配合异步 I/O 驱动。两端的 API 交互严格遵循 OpenAPI 3.0 规范，通过 Swagger UI 自动生成交互式接口文档。

2 主要技术

2.1 前端技术栈

(1) Vue3

Vue.js 是一款用于构建用户界面的 JavaScript 框架。它基于标准 HTML、CSS 和 JavaScript 构建，并提供了一套声明式的、组件化的编程模型，能够高效地对用户界面进行开发^[2]。Vue3 作为 Vue.js 最新的主要版本，提供了更高效的虚拟 DOM 算法和更灵活的 Composition API，能够显著提升开发效率和应用性能。

在本项目中，使用 Vue 3 的 Composition API 构建了复杂的组件逻辑，同时使用组件地封装技术，使得前端代码具有更好的可读性，同时增加了前端代码地健壮性，便于后续进行维护与进一步开发^[7]。

使用 Vue.js 可以去除繁琐的 DOM 操作，只需要关注数据的源头，而不用担心 DOM 元素变化之后的绑定变化。这就是数据驱动与 DOM 驱动的差别^[11]。

本项目前端采用单页面应用技术进行开发。单页面应用，加载单个页面，并在用户与应用程序交互时动态更新该页面，浏览器一开始会加载需要的 HTML、CSS 和 JavaScript。相较于无论是页面有数据刷新，还是点开新选项卡的情况，都要浪费资源重新加载全部页面元素，而 Vue.js 框架的单页面应用技术可以解决在这过程中页面不断重新加载的问题。

(2) TypeScript

TypeScript 是由微软开发的一种编程语言，它是 JavaScript 的超集，这意味着它包含了 JavaScript 的所有功能，并在此基础上添加了新的特性和工具。TypeScript 的设计目标是帮助开发者构建大型应用程序，并且它的代码最终会被编译成纯 JavaScript，使得编译后的代码可以在任何浏览器、任何计算机和任何操作系统上运行。Typescript 如今在 Web 前端领域使用率发展飞速，成为 Web 前端的强势新规范^[12]。通过使用 TypeScript，开发者可以提高代码质量、提升代码可维护性、提高开发效率，并更好地管理大型项目。

本项目中，TypeScript 作为前端地主要开发语言，用于实现主要的前端组件业务逻辑，使代码更加模块化和可重用。

（3）ElementPlus

Element Plus 是一个基于 Vue 3 的高质量 UI 组件库。它包含了丰富的组件和扩展功能，例如表格、表单、按钮、导航、通知等，让开发者能够快速构建高质量的 Web 应用。Element Plus 的设计理念是：提供开箱即用的 UI 组件和扩展功能，帮助开发者快速构建应用程序，同时提供详细的文档和教程，让开发者更好地掌握和使用 Element Plus。

ElementPlus 有四大设计原则：一是一致，即所有的元素和结构需保持一致，页面逻辑与现实生活的流程、逻辑保持一致；二是反馈，即通过界面样式和交互动效让用户可以清晰的感知自己的操作，操作后，通过页面元素的变化清晰地展现当前状态；三是效率，即操作流程需简洁直观，语言表达需清晰明确，界面需简单直白；四是可控，即根据场景可给予用户操作建议或安全提示，但不能代替用户进行决策，并且用户需要能自由的进行操作，不受限制^[3]。这一切都旨在给用户更好的体验。

在本项目中，ElementPlus 的组件被用于构建出一个美观的、现代化的前端用户交互页面，提升用户交互体验。

（4）Echarts

ECharts 是一个使用 JavaScript 实现的开源可视化库，可以流畅的运行在 PC 和移动设备上，为前端提供直观，交互丰富，可高度个性化定制的数据可视化图表^[4]。ECharts 最初是由百度团队开发的，后开源并被捐赠给 Apache 软件基金会（ASF），成为了 ASF 的顶级孵化项目之一。它除支持统计工作中经常用到的折线图（区域图）、直方图（条状图）、散点图（气泡图）、K 线图、饼图（环形图）、雷达图（填充雷达图）外，还提供了大数据分析中使用的和弦图、力导向布局图、地图、仪表盘、漏斗图等图表^[10]。

本项目中，前端页面使用 ECharts 构建可交互式图表，利用其提供的多种图表类型直观展示项目涉及的复杂数据，并且 ECharts 在处理大量数据时性能良好，具有极其丰富的配置选项，能够根据具体需要灵活调整。

2.2 后端技术栈

（1）FastAPI

FastAPI 是一个现代、高性能的 Python Web 框架，基于标准 Python 类型提示，能够自动生成 API 文档。它具有高性能、高效编码、智能、简单、标准化等特点，能够快速搭建项目所需要的后端服务，符合当前三周完成前后端的高负担任务需求场景^[1]。

在本项目中，FastAPI 被用于构建 RESTful API 接口，为前端提供数据交互服务。通



过 Pydantic 模型，对数据进行了严格的类型验证，确保数据的正确性和安全性。

（2）MariaDB

MariaDB Server 是一个通用的开源关系数据库管理系统，它保持与 MySQL 的高度兼容性，并且大多数使用 MySQL 的流行应用程序将与 MariaDB 无缝协作，提供了丰富的功能和良好的性能。

在本项目中，MariaDB 被用于存储医保数据，包括参保人信息、就医记录、费用明细等。通过 SQLAlchemy ORM，实现了对数据库的便捷操作，同时支持异步查询，提高了系统的响应速度。

（3）Redis

Redis 是一个开源，内存存储的数据结构服务器，可作为 key-value 型的非关系型数据库，支持数据的持久化，可以将内存中的数据保持在磁盘中，重启的时候可以再次加载进行使用。其具有高性能、操作原子性等特点。

本项目中 Redis 数据库因为其支持 key 过期等特性以及非关系型数据库的特点，作为本项目存储验证码使用的数据库。

（4）SQLAlchemy

SQLAlchemy 作为 Python 生态中功能完备且高度灵活的数据库工具包，在本项目中扮演着核心数据访问层的角色。它通过两大核心模块为不同开发场景提供支持：对象关系映射（ORM）模块允许开发者使用 Python 类和方法直观地操作数据库实体，将复杂的 SQL 交互抽象为面向对象的编程范式，大幅提升代码可读性和可维护性；而其核心（Core）模块则提供更底层的 SQL 表达式语言和连接管理能力，为需要精细控制 SQL 语句或追求极致性能的场景提供了轻量级解决方案^[5]。

得益于其跨平台设计，SQLAlchemy 支持包括 PostgreSQL、MySQL、SQLite 和 Oracle 在内的主流数据库后端，并通过统一的 API 接口屏蔽了不同数据库方言的差异。这一特性使项目具备极强的环境适应性和扩展潜力，无论是本地开发环境使用的 SQLite，还是生产环境部署的高性能 PostgreSQL 集群，均能无缝切换而无需重写数据访问逻辑。

本项目中，SQLAlchemy 用于与数据库进行数据交互，同时兼容异步操作，满足 FastAPI 后端对于异步处理数据的需求，方便与 FastAPI 集成，通过依赖注入实现高效的数据库操作。

3 需求分析

3.1 功能需求

(1) 系统功能组成

本医保反欺诈系统由四大核心功能模块构成：用户登录模块为医保人员提供系统访问入口；整体数据分析模块专注于宏观层面，提供包括整体样本数量统计、异常样本比例分析、欺诈类型分布展示以及整体欺诈概率评估在内的综合洞察；个人数据分析模块则聚焦于微观个体案例，支持对个人欺诈类型识别、详细药费组成分析以及个体欺诈概率的评估；最后，欺诈数据处理模块提供对识别结果的管理功能，涵盖自费占比分析、欺诈类型编辑以及对欺诈记录的删除等关键操作。这四个模块协同工作，实现了从数据概览到个案深挖，再到结果管理的完整反欺诈闭环流程。

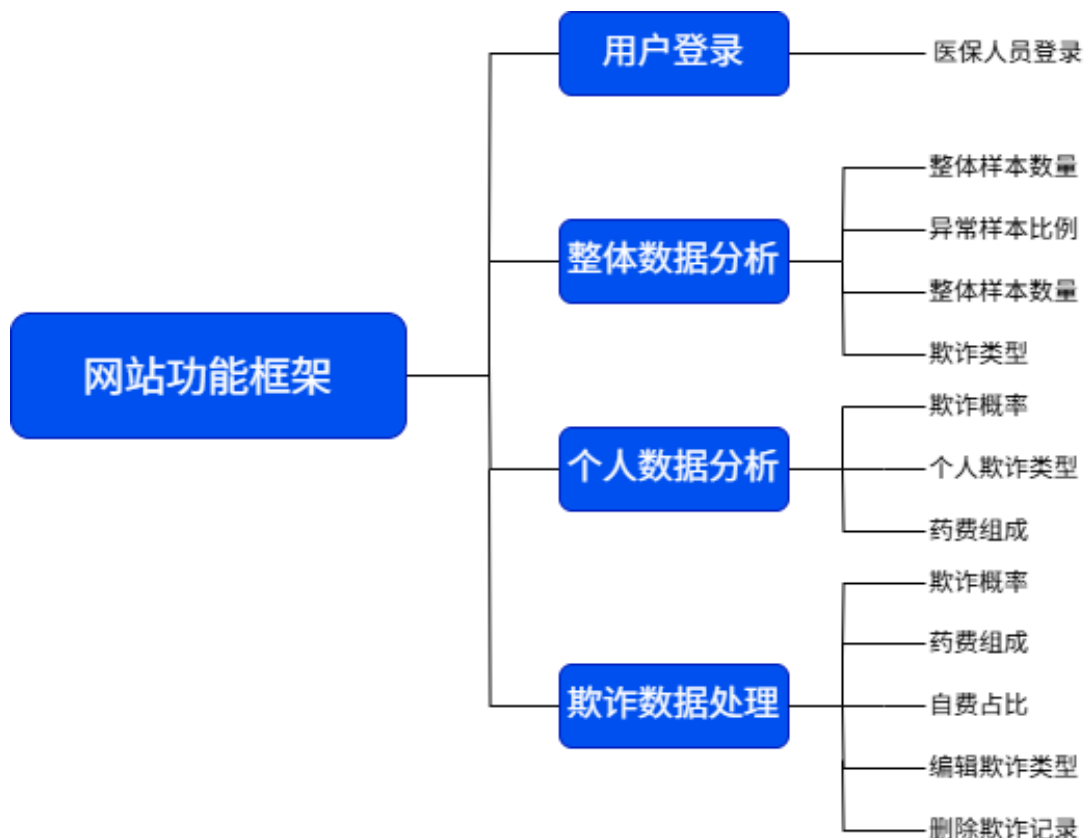


图 3.1 系统功能组成图

(2) 数据管理

数据管理模块采用分层架构设计，实现医保全流程数据的全生命周期管理。系统支

持通过标准接口批量导入参保人员基础信息、就诊记录明细、医疗费用清单等结构化数据，导入过程中自动执行数据清洗和标准化处理，包括去重、格式转换、异常值修正等。针对数据查询功能，系统提供多维度组合检索能力，可按照参保人 ID、就诊日期范围、医疗机构、疾病诊断代码等关键字段进行精确或模糊查询，查询响应时间控制在 500 毫秒内。

数据更新机制采用差异同步策略，当检测到数据变更时，系统自动记录变更内容并生成版本快照，支持数据回滚操作。删除操作实施逻辑删除与物理删除双机制，普通用户执行删除时系统仅标记删除状态，超级管理员可进行物理清除。所有数据操作均记录完整审计日志，包括操作人员、时间戳、修改前后数据差异等关键信息。

（3）欺诈检测与分析

欺诈检测系统构建了多模型融合的智能分析体系。底层采用 Spark MLlib 实现的随机森林算法对就诊行为特征进行初步筛查，中层使用深度学习模型捕捉非线性的复杂欺诈模式，顶层通过规则引擎执行医保政策合规性校验。系统每小时自动生成全量数据的风险评分，并按参保人维度聚合展示欺诈概率分布。

（4）可视化与展示

数据可视化平台采用响应式设计，自动适配不同终端设备。核心仪表盘整合了欺诈风险全景视图，通过地理热力图展示区域风险分布，使用时序折线图呈现欺诈趋势变化，借助桑基图分析欺诈类型转化路径。针对个体参保人，系统提供 360° 画像视图，整合基础属性、就医轨迹、费用构成、关联网络等多维度信息。

（5）名单管理

异常名单管理系统实施分级预警机制，将风险人员划分为观察名单、重点关注名单和高风险名单三个等级。系统支持基于多条件的智能名单生成，如"同一医生接诊超过 10 次且费用超标 200%的参保人"。编辑操作实施四眼原则，普通审核员的修改需经过主管复核确认才能生效。系统自动追踪名单变更历史，建立完整的证据链，包括修改依据、审批流程、相关佐证材料等。对于确认的欺诈案例，可一键加入行业共享黑名单系统。

（6）智能问答

AI 问答系统基于微调的医保专业大语言模型，构建了包含政策法规、典型案例、审核要点等内容的百万级知识库。系统支持多轮对话理解，能准确解析"如何识别分解住院骗保行为"等专业问题。针对复杂查询，助手可自动生成结构化分析表格，并标注相关政策依据。



智能引擎实时监控医保局最新政策动态，每周自动更新知识库。问答过程支持语音输入和播报，方便现场稽核人员使用。所有咨询记录均存入案例库，用于持续优化模型效果。

3.2 非功能需求

（1）性能

系统需具备高效处理能力，以满足大量用户同时访问的需求。预测在业务高峰期，系统需能稳定承载每秒数百至数千次的并发 API 请求。核心数据查询操作的响应时间需严格控制在亚秒级，复杂分析操作的响应时间也应控制在可接受范围内，避免用户长时间等待导致体验下降。系统需具备处理海量医保数据的能力，确保数据导入、批量更新及后台分析任务不会显著阻塞前台用户交互或耗尽系统资源。

（2）安全

鉴于医保数据的极端敏感性和隐私保护法规的严格要求，系统必须建立全方位的安全防护体系。这包括：强制性的严格身份认证与授权机制，确保只有经过验证且被明确授权的用户才能访问特定功能和数据，实施基于角色的最小权限原则；端到端的数据加密，要求在数据传输和存储过程中对敏感个人信息（PII）和医疗健康信息（PHI）进行强加密保护；完备的安全审计与追溯能力，系统需记录所有关键操作的详细日志，以便进行安全审计和责任追溯；防御常见攻击手段，系统需具备抵御如 SQL 注入、跨站脚本（XSS）、跨站请求伪造（CSRF）等常见网络攻击的能力。

（3）可扩展性

系统架构需具备足够的灵活性与扩展性，以适应未来业务增长和需求变化。这要求：功能模块化，新的欺诈分析模型、数据处理规则或可视化组件应能相对独立地添加或更新，而不影响系统核心稳定性；数据模型可扩展，当出现新的欺诈模式或需要追踪新的数据指标时，数据存储结构应能支持便捷的扩展；处理能力可伸缩，系统需能在数据量激增或分析复杂度提高时，通过增加计算或存储资源来维持性能水平；支持定制化规则，系统需提供机制允许管理员或特定角色根据本地政策或特定场景，配置或调整欺诈检测的规则、阈值和逻辑。

（4）易用性

考虑到最终用户可能并非技术专家，系统界面与交互设计必须高度用户友好和直观：界面简洁清晰，布局逻辑清晰，避免信息过载，核心功能应易于发现和访问；数据可视

化直观易懂，使用图表展示复杂数据时，应选择最合适的类型并清晰标注，避免歧义，配色需专业且符合无障碍标准；交互流程顺畅自然，操作步骤应尽可能简化，提供明确的引导、反馈和错误提示，减少用户的认知负担和操作失误；术语通俗化，尽量避免过度技术性术语，使用医保业务领域内通用的、用户熟悉的语言进行表述，或在必要时提供简洁的术语解释。整体目标是为用户提供高效、低学习成本、无挫败感的操作体验。

4 总体设计

4.1 项目总体架构

本项目采用前后端分离架构，前端基于 Vue3+TypeScript 和 Element Plus 组件库构建可视化界面，实现数据看板、欺诈分析和 AI 交互等功能；后端使用 Python 开发，通过 RESTful API 提供核心服务，数据层采用 SQL 主数据库与 Redis 缓存协同管理，形成高内聚、低耦合的模块化系统。项目总体架构如图 4.1 所示。

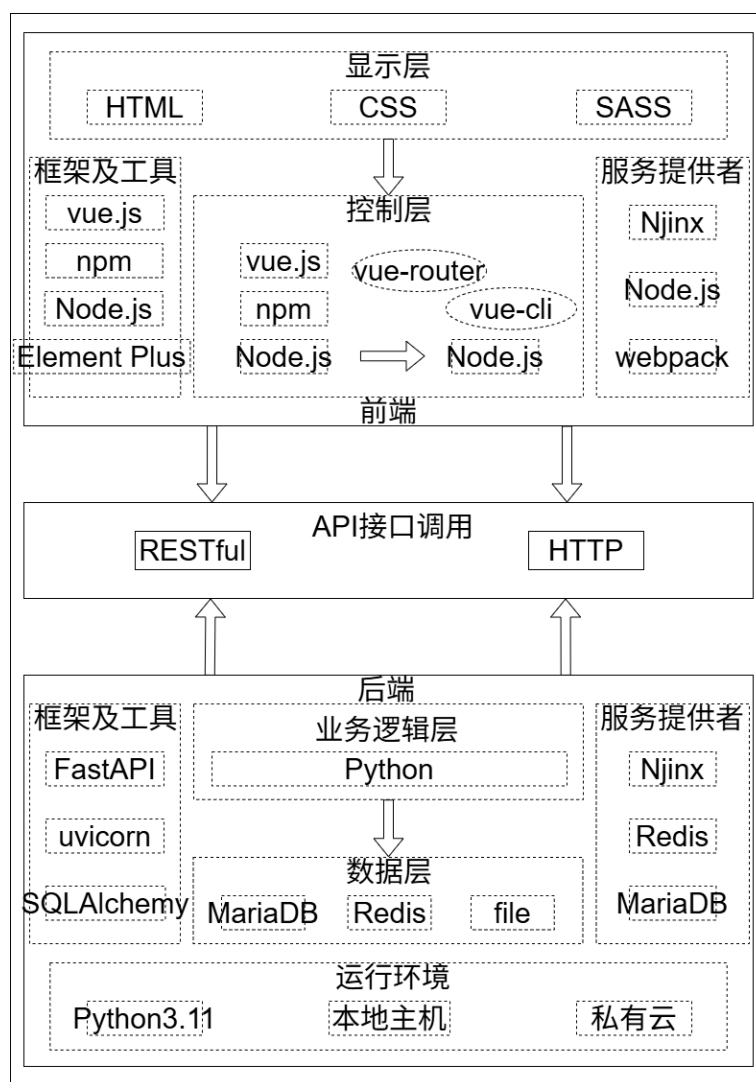


图 4.1 项目总体架构图

4.2 项目功能模块

4.2.1 用户登录模块

该模块提供用户注册、登录、身份验证等功能，确保系统访问安全。用户若没有系

统账户可以进行注册，登录提供用户名密码登录和邮箱验证码登录两种，方便用户进行选择。界面样式如图 4.2 所示。

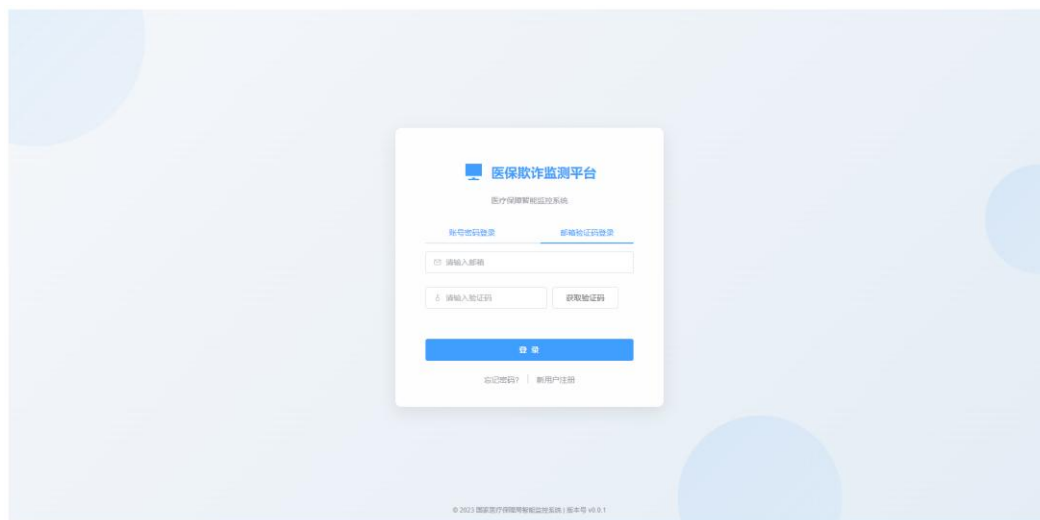


图 4.2 用户登陆界面

4.2.2 数据总览模块

该模块集中展示了医保数据分析的总体分析结果，包括当前数据库中数据条数，正常/异常数据条数以及异常数据在总数据中的占比。页面中还使用了图表展示了项目人工智能模型进行数据分析所使用的特征因子以及数据库中所判断并完成分类的几种欺诈类型。该模块能够方便用户快速了解项目数据分布。界面样式如图 4.3 所示。



图 4.3 数据总览

4.2.3 数据分析模块

该模块逐条展示了分析之后的医保信息，包含记录的欺诈概率，欺诈类型以及医疗

资金的分布。用户可以在这个模块方便地查看每一条记录的分析结果。界面样式如图 4.4 所示。

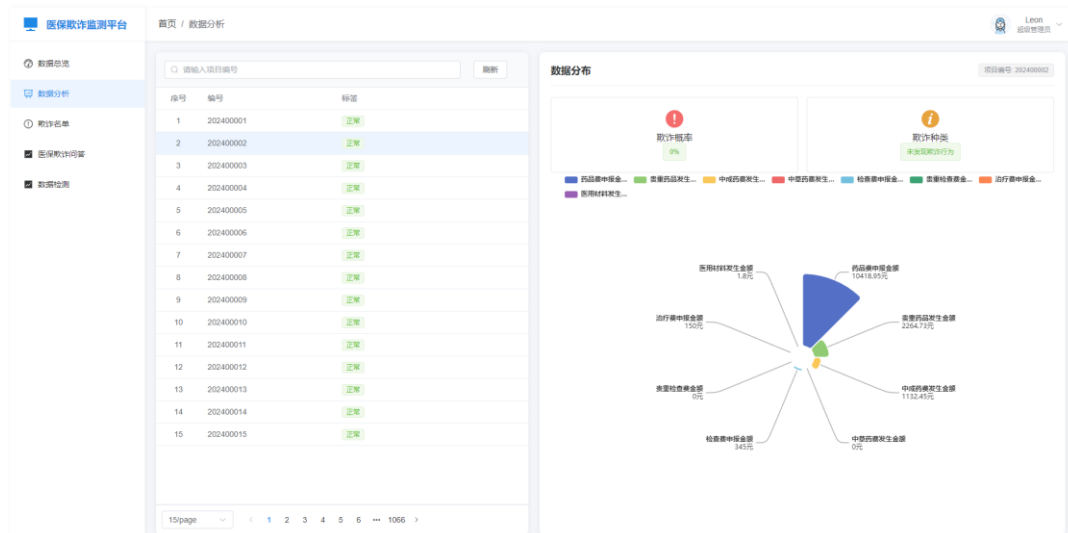


图 4.4 数据分析

4.2.4 欺诈名单模块

该模块集中展示了分析结果中存在欺诈概率的数据，点击列表中的任意一行数据便可查看它的详细分析结果，也可以点击修改或是删除来进行进一步操作。界面样式如图 4.5 所示。

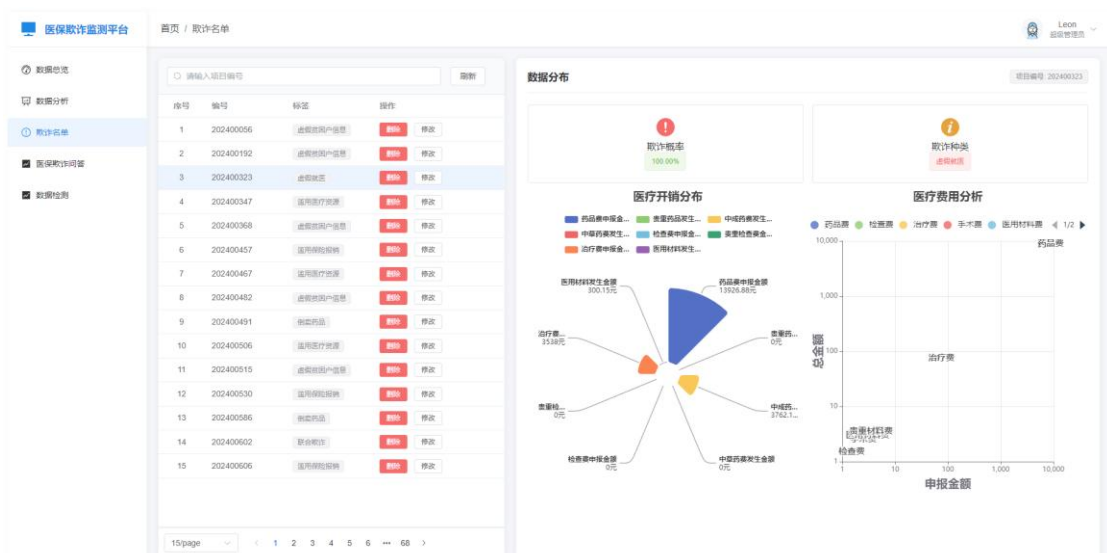


图 4.5 欺诈名单

4.2.5 医保欺诈问答模块

该模块接入了微调的大语言模型,为用户提供医保欺诈相关的专业知识的科普和解答。

答。相比较于传统搜索，大语言模型能提供更易理解的回答，提供更个性化的服务。界面样式如图 4.6 所示。



图 4.6 医保欺诈问答

4.2.6 数据检测模块

此模块利用 SparkMLlib 创建流分析模型，支持用户输入医保数据，系统能进行处理，并输出分析结果。界面样式如图 4.7 所示。

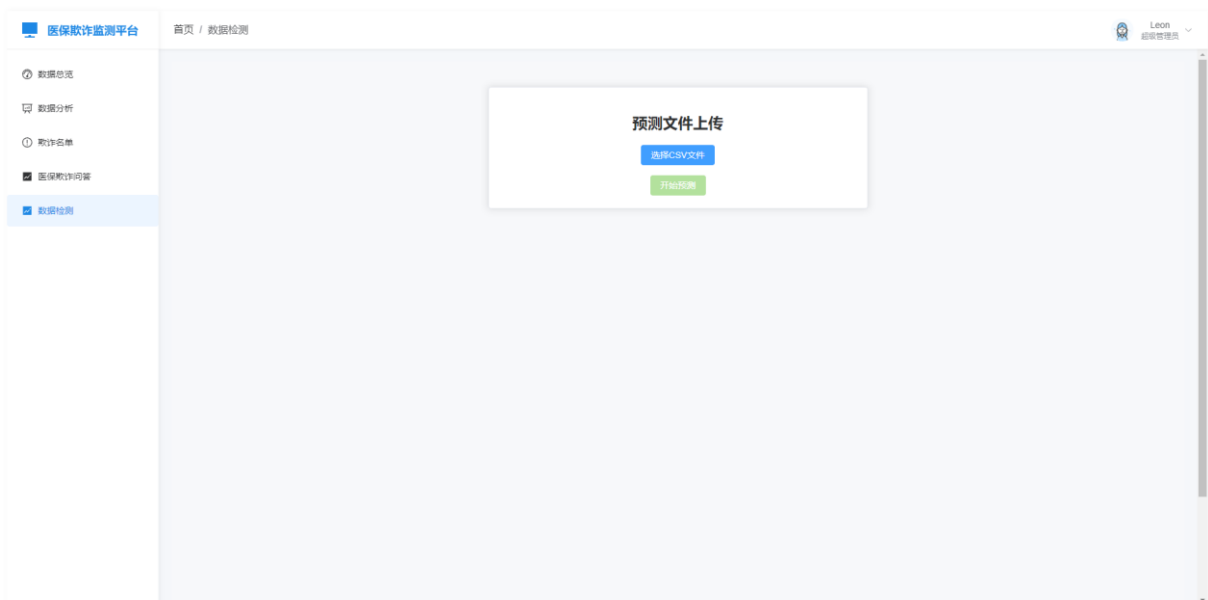


图 4.7 数据检测



5 详细设计与实现

5.1 数据库连接

FastAPI 有自己完整的生命周期方法，能够准确定义服务器启动之前或是之后所执行的方法。本系统涉及与数据库进行连接和数据传输，因此需要配置数据库连接文件，并在 FastAPI 的 main.py 文件中在正确的生命周期中调用，方能正确地连接数据库。

5.1.1 连接 MariaDB 数据库

项目使用 SQLAlchemy 三方库来连接数据库并对数据库进行数据读写。SQLAlchemy 作为一个功能强大的三方库，拥有非常多的方法。首先要创建一个数据库初始化方法。定义全局变量 engine 用于存放数据库连接对象，根据 FastAPI 的需求，定义异步连接，然后定义要连接的数据库的 URL，最后创建连接，即可连接到数据库。连接数据库逻辑如图 5.1 所示。

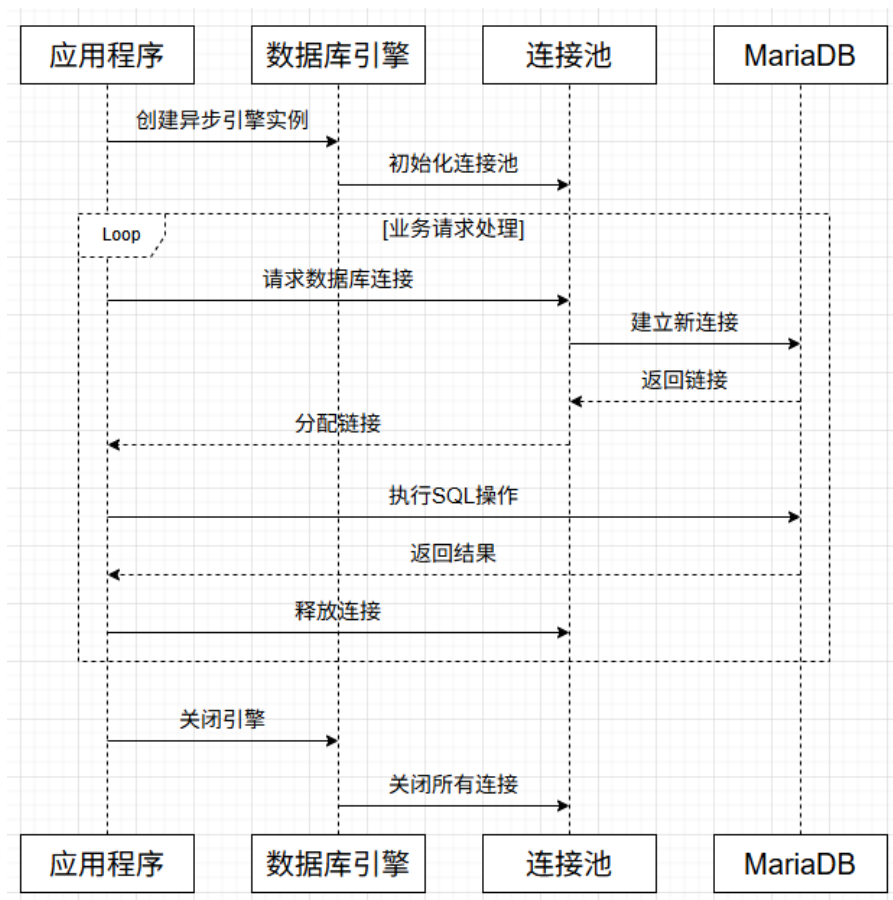


图 5.1 连接 MariaDB 时序图

针对 FastAPI 的异步会话需求，需要定义一个异步上下文管理器，用于安全地管理 SQLAlchemy 的异步数据库会话。它能自动管理数据库会话的生命周期（如创建、提交、回滚、关闭等），能够安全、高效地处理 SQLAlchemy 的异步会话。

最后，需要处理数据库的关闭。只需要调用 SQLAlchemy 的 dispose 方法即可快速关闭，避免出现错误。

5.1.2 连接 Redis 数据库

本项目中涉及到使用 Redis 来保存验证码和实现验证码的生命周期管理。项目中使用 redis.asyncio 库来对 Redis 的连接进行管理。连接数据库逻辑如图 5.5 所示。

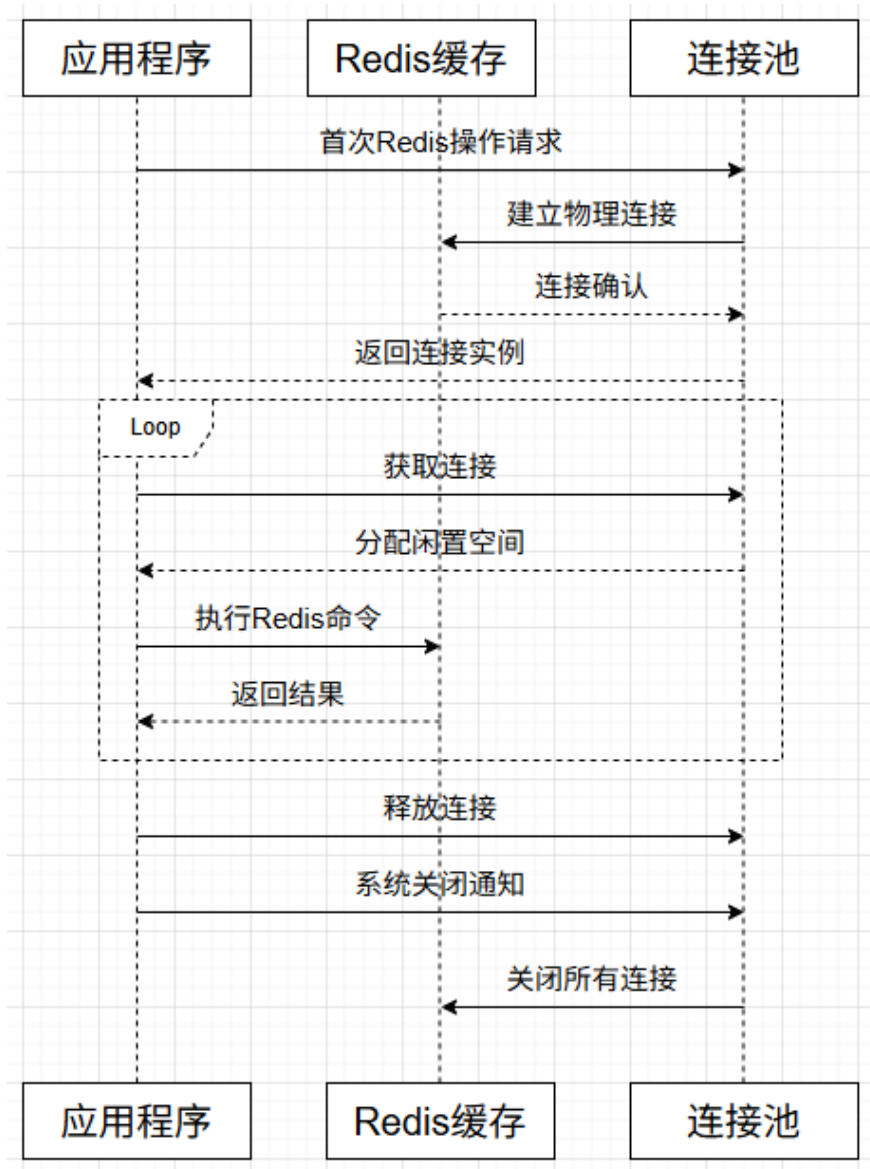


图 5.2 Redis 数据库连接时序图

在创建 Redis 连接池时，使用懒加载技术，因为不是每一次使用系统都会用到 Redis，

因此设计为只有需要向 Redis 中传输数据时，才会创建 Redis 连接池，且所有的 Redis 连接都共用一个 Redis 连接池，这样可以降低开销，避免频繁创建/销毁连接，提升性能。

最后，创建一个关闭连接方法，以便在关闭服务时能够顺利关闭 Redis 连接池，适配异步操作，避免报错。

5.2 用户登录

登录模块主要目的为限制未经授权的人员使用此系统，达到防止系统内数据泄露或是系统被异常使用的情况。用户登录时的数据路径如图 5.3 所示。

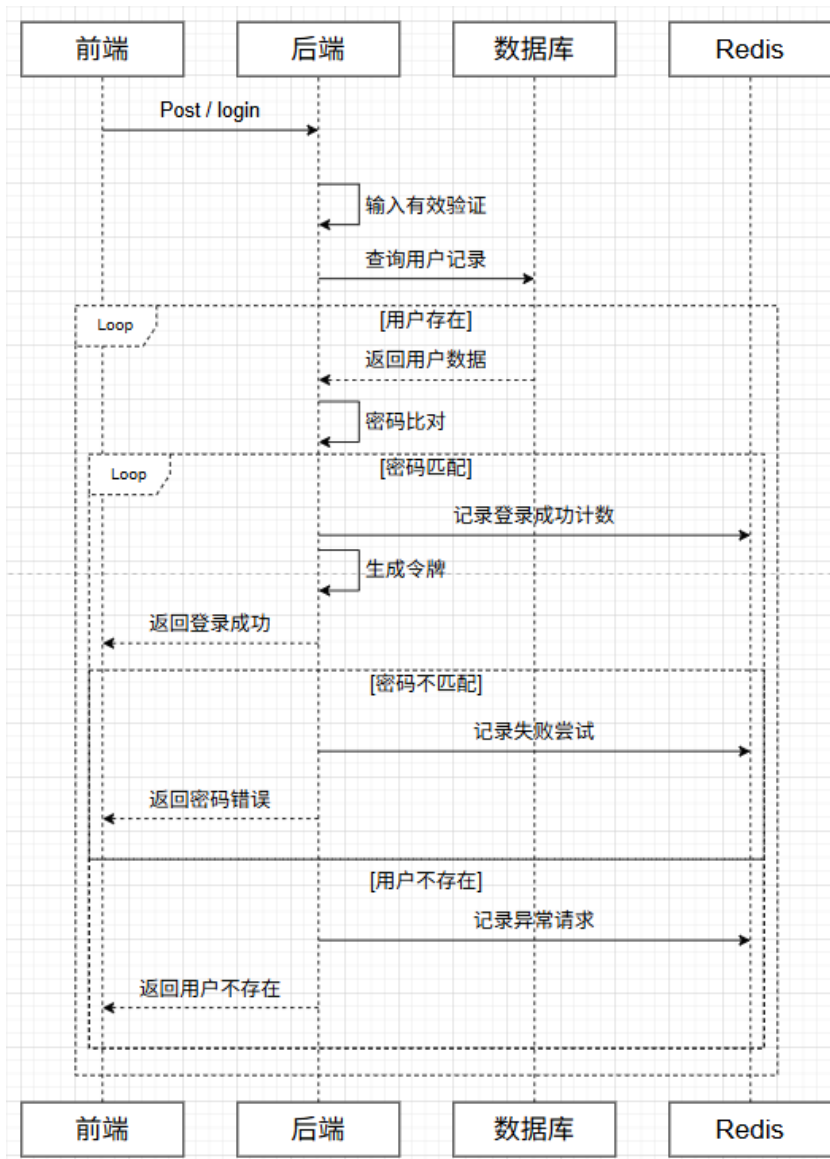


图 5.3 用户登录逻辑时序图

本系统采用多层防御机制构建了安全可靠的认证体系。当用户在登录界面输入邮箱和密码后，系统立即启动严谨的验证流程。前端首先对输入内容进行即时校验，通过正则表达式确保邮箱格式符合国际标准，同时对密码进行复杂度分析，要求必须包含大小

写字母、数字且长度在 8-32 位之间，这些实时校验避免了无效请求直接冲击后端服务。

验证通过的凭证通过 HTTPS 加密通道传输到后端服务器，请求体采用 JSON 格式封装，并附加 CSRF 令牌等安全头信息。后端接收到请求后，首先在 Redis 中检查该账户的登录尝试记录，如果短时间内失败次数超过阈值，则直接拒绝请求并提示账户暂时锁定。对于正常请求，系统会查询数据库获取对应用户记录，这个查询过程通过 email 字段的唯一索引优化，确保在高并发情况下仍能快速响应。

密码比对环节采用 bcrypt 哈希算法，这种特意设计缓慢的算法能有效抵御暴力破解。系统会严格比较用户输入密码与数据库存储的哈希值，任何细微差异都会导致认证失败。为了防止时序攻击，无论密码是否正确，系统都会执行完整的哈希计算过程，确保响应时间一致。对于认证失败的请求，系统会在 Redis 中累加失败计数器，并记录详细的尝试日志，这些数据既用于实时防护，也为后续安全审计提供依据。

成功通过认证的用户，前端会将其用户名和昵称使用 LocalStorage 存放在浏览器中，方便用户多次登录。同时，系统会记录本次登录的 IP 地址、设备特征等上下文信息，用于后续的异常登录检测。在整个认证过程中，所有关键操作都会生成详细的审计日志，包括请求时间、处理结果、异常情况等，这些日志通过异步方式写入专用存储，既不影响主流程性能，又满足安全合规要求。

对于各种异常情况，系统设计了人性化的错误提示机制。当用户输入错误的密码时，不会直接告知是邮箱还是密码错误，而是使用统一的模糊提示。登录失败时的提示如图 5.4 所示。

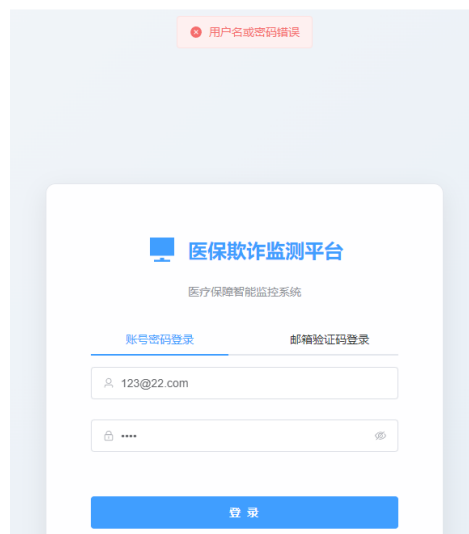
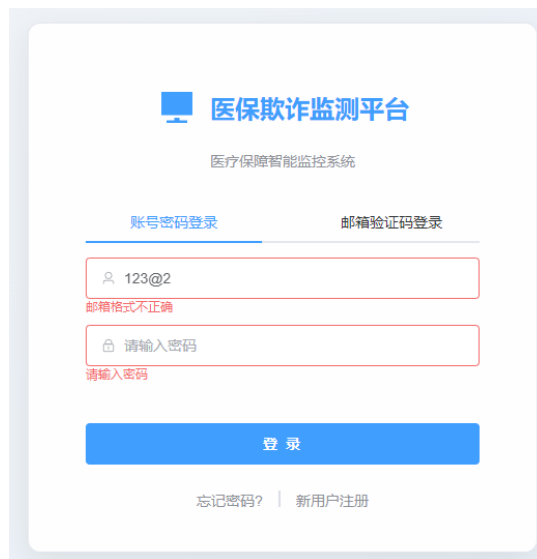


图 5.4 登录失败

同时，为了预防用户输入非常规的用户名或密码导致系统出现不可预知的问题，系

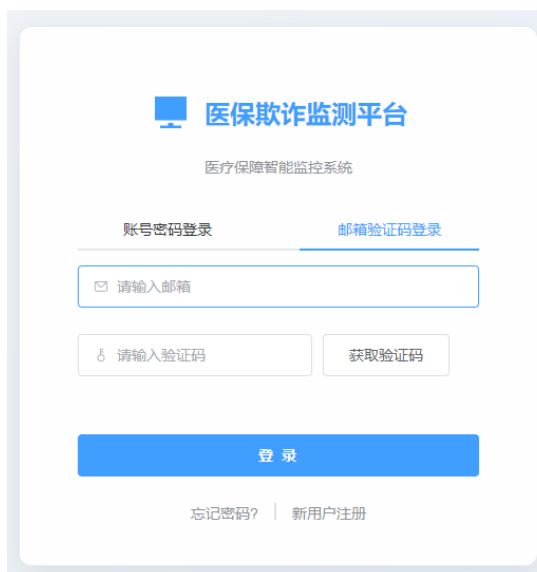
统对于用户在输入框中输入的内容进行了类型限制，若用户输入内容不合规范，输入框会进行提示。异常输入提示如图 5.5 所示。



The screenshot shows the 'Medical Insurance Fraud Monitoring Platform' login page. It has two tabs: 'Account Password Login' and 'Email Verification Code Login'. The 'Email Verification Code Login' tab is active. The email input field contains '123@2' and shows a red error message '邮箱格式不正确' (Email format is incorrect). The password input field is empty and shows a red error message '请输入密码' (Please enter password). Below the input fields is a blue '登录' (Login) button. At the bottom, there are links for '忘记密码?' (Forgot password?) and '新用户注册' (New user registration).

图 5.5 输入异常提示

为防止用户忘记了密码无法登录，系统提供使用邮箱和验证码登录的功能。用户输入邮箱之后，点击获取验证码按钮之后，后端会接收到邮箱数据，从数据库中检索是否有邮箱数据，存在则会正常发送验证码，若不存在则会返回错误，前端对用户进行提示“邮箱未注册”，验证码存在 5 分钟的使用时限，且为了预防因为用户的频繁访问导致系统卡顿，用户获取验证码必须有一分钟的间隔。点击获取验证码后会进入倒计时无法点击，且后端也会进行检测请求是否过于频繁。验证码登录界面如图 5.6 所示。



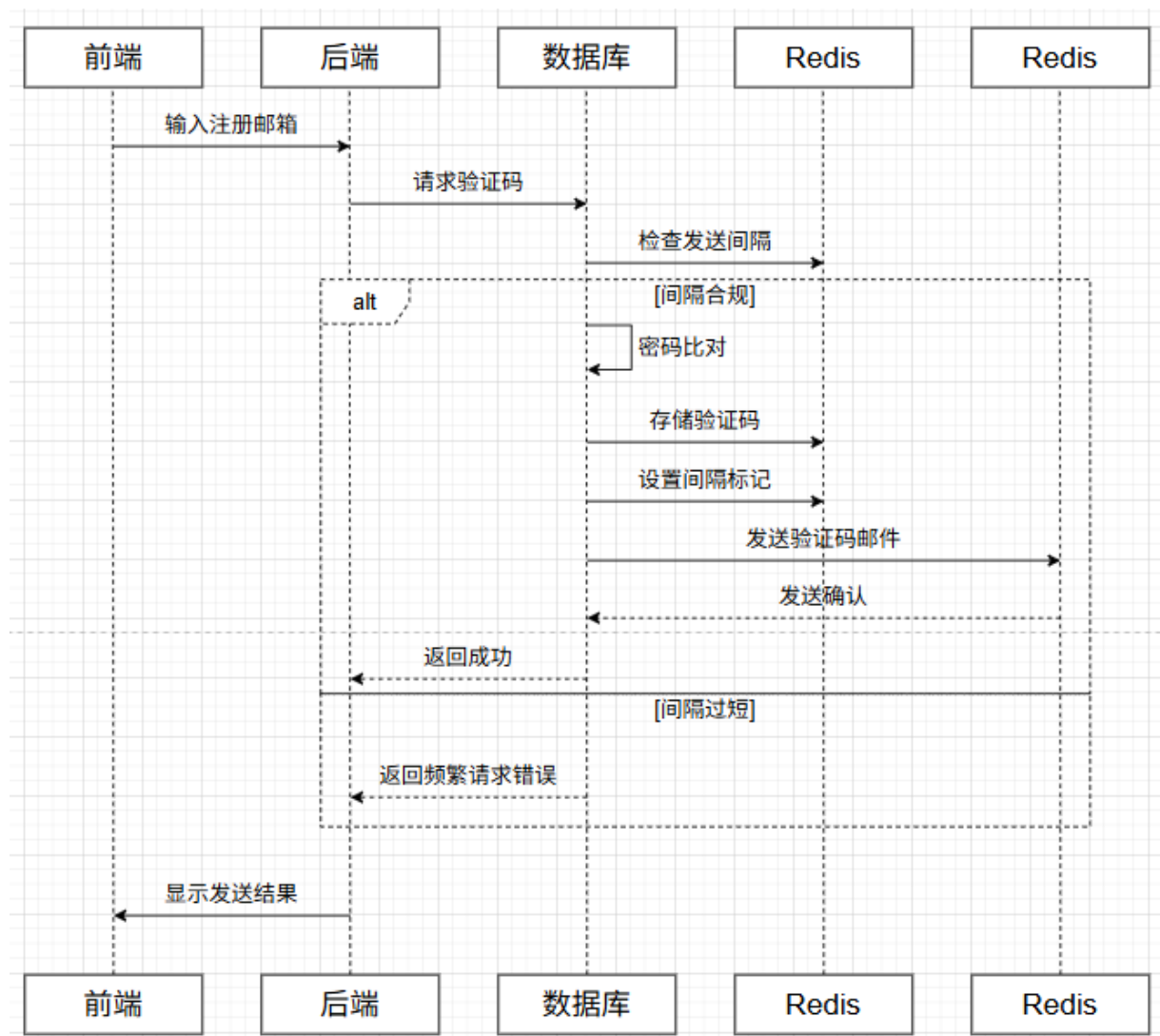
The screenshot shows the 'Medical Insurance Fraud Monitoring Platform' login page. It has two tabs: 'Account Password Login' and 'Email Verification Code Login'. The 'Email Verification Code Login' tab is active. The email input field contains '请输入邮箱' (Please enter email). Below it is a '请输入验证码' (Please enter verification code) input field and a '获取验证码' (Get verification code) button. Below these is a blue '登录' (Login) button. At the bottom, there are links for '忘记密码?' (Forgot password?) and '新用户注册' (New user registration).

图 5.6 邮箱验证码登录

验证码使用随机数生成随机的 6 位验证码，为实现验证码 5 分钟的生命周期和过于



频繁请求的检测，后端使用 Redis 数据库来对验证码进行存储。Redis 数据库可以使用 key-value 进行存储，可以定义存储记录的生命周期。请求验证码成功后，数据库中会存储验证码本身和其对应的时间标记，分别有 5 分钟和 1 分钟的生命周期，用户请求验证码时，若存在对应邮箱的验证码时间标记，则会阻止这次请求。登陆成功后便会删除数据库中对应的验证码数据和标记数据。验证码登录逻辑如图 5.7 所示。



5.7 验证码登录时序图

若用户没有账号，则可以进行注册。注册时需输入邮箱、密码以及用户自定义的昵称。为了验证邮箱的有效性，同样需要获取验证码。验证码的获取和发送逻辑复用了登录的验证码逻辑，避免过多的代码重复，降低代码的大小。注册弹窗如图 5.8 所示。



新用户注册

✕

✉ 请输入邮箱

🔒 请输入密码

👤 请输入昵称 (不超过5个字)

📄 请输入验证码

获取验证码

取消 注册

图 5.8 用户注册弹窗

为防止存在人员直接使用对应 URL 直接进入系统绕过登录界面，同时方便用户的二次登录，系统使用了 vue-router 的路由守卫结合 Local Storage 进行判断。成功登录时，系统会将用户数据存储在浏览器的 Local Storage 中。LocalStorage 是 Web 存储 API 的一部分，它允许网站在用户的浏览器中存储键值对数据。存储在 LocalStorage 中的数据没有过期时间，即使浏览器关闭后数据依然会保留，直到被明确地删除。

5.3 数据总览

数据总览模块使用多种可视化方法整体地体现了系统中的数据的结构和组成。

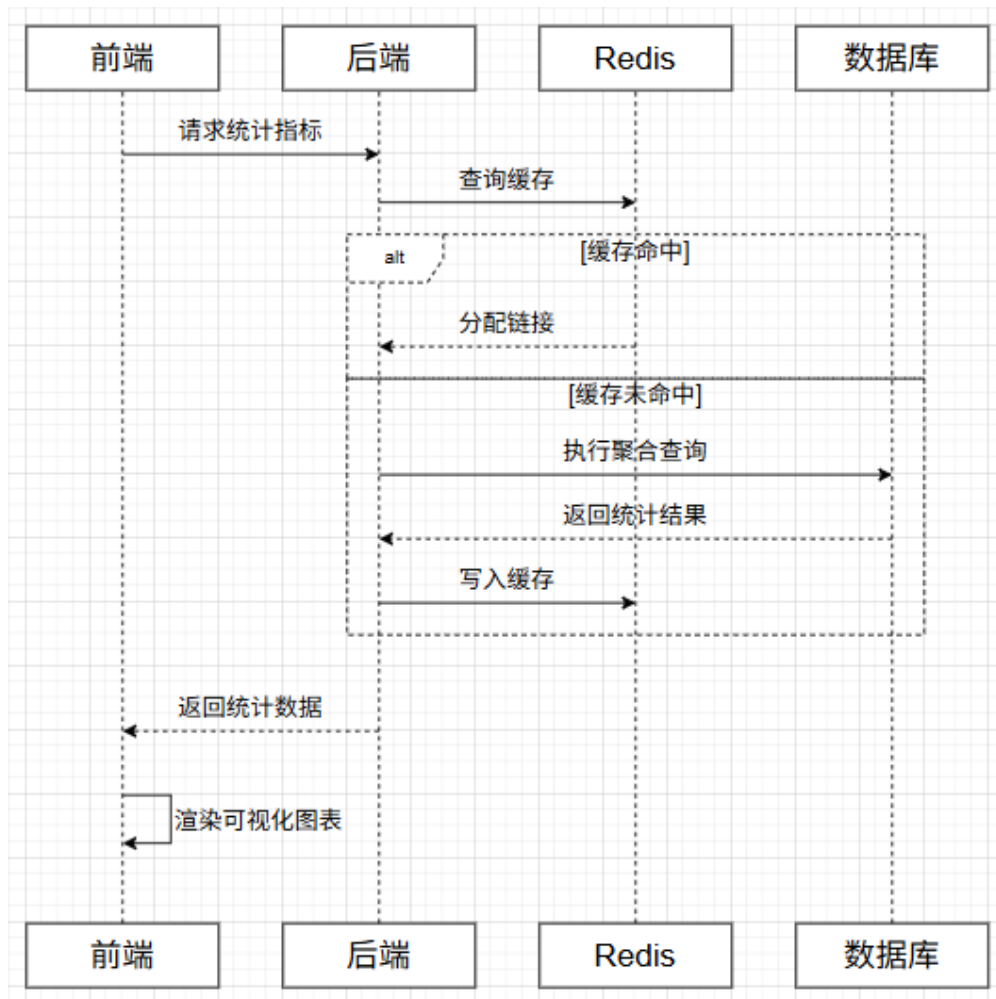
页面上半部分使用卡片展现了当前系统中数据的分布情况，展现当前数据库中的数据存在多少的正常和异常数据，同时计算出异常数据所占整体数据的比例。用户从该模块中便可以了解数据集的组成，辅助自己对于系统数据的判断和使用。数据卡片如图 5.9 所示。



图 5.9 数据卡片

数据库中的数据为单条记录，没有统合的数量记录。因此，后端需要编写逻辑实现记录条数的计算功能。后端使用 SQLAlchemy 三方库读取所有数据，然后使用库中的 select 函数优化代码性能。由于涉及到大量数据的计算处理，可能会出现延迟，所以需要使用异步处理，等待从数据库拉取数据结束之后再进行接下来的计算操作，避免出现因为数据拉取卡顿导致的系统卡顿，增加用户的使用体验，同时页确保了整个系统的

流畅性与稳定性。数据查询逻辑如图 5.10 所示。



5.10 数据查询逻辑时序图

下半部分使用了两张 ECharts 图表，图表样式如图 5.11 所示。环形图用于展示欺诈判断模型所依据的数据特征因子，因子再图中所占的环越大，说明其再评判中重要性越高。而散点图则是使用气泡图来较为清晰且美观的方法展现了数据库中分析出来的所有欺诈类型。

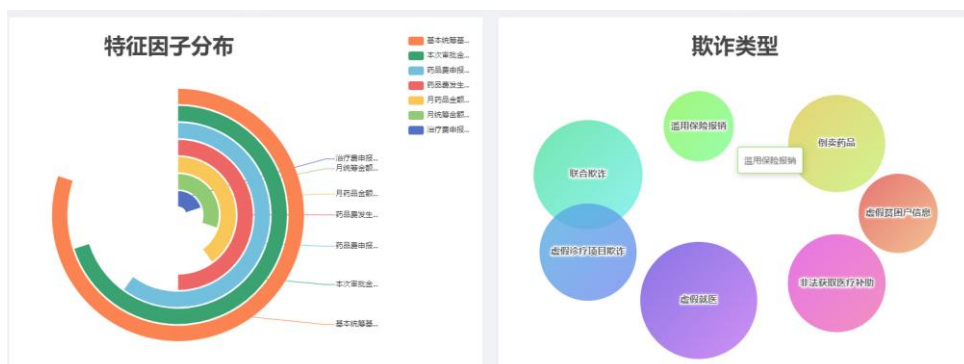
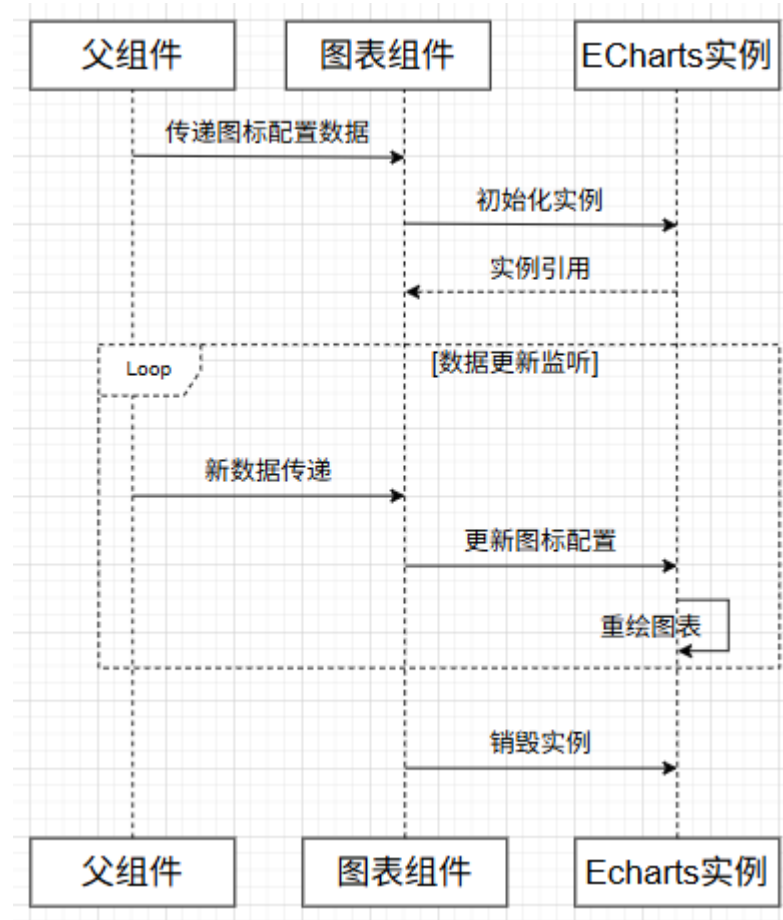


图 5.11 类型图

图表渲染数据路径如图 5.12 所示。



5.12 图表渲染时序图

由于涉及到较多的图表，若全部放在一个页面中则会显得非常杂乱，因此在项目中大量利用了 vue3 的组合式 api，将部分页面内容封装成组件。

Vue3 组合式 API 主要用于在大型组件中提高代码逻辑的可复用性和可维护性，通过在 `setup` 函数中按逻辑关注点对代码进行分组，提取逻辑片段并与其他组件共享，从而编写更有条理的代码。组合式 api 使用 `setup` 钩子作为在组件中使用组合式 API 的入口，它能将原本使用选择式 api 时需要分开定义的方法与变量集中打包在一起定义，随后统一将其暴露给父组件使用。在 vue3 中，能够通过 `<script>` 标签中添加 `setup` 作为一种更加高效地使用组合式 API 的语法糖，免去显示声明 `setup()` 函数，所有在 `<script setup>` 中定义的变量、方法、引入的组件等，都会自动暴露给模板使用。这样的使用方法能在 TypeScript 下有更好的类型推断和自动补全体验，还能简化代码，更方便地使用 Vue 3 的组合式 API，提升开发效率。

在页面样式部分，在 `<style>` 标签后添加 `scoped` 关键字使得该组件中定义的样式被封装，即所有定义在该组件中的样式只能在该组件中被使用，在其它的地方是不能被使

用的。这样的定义方法避免了各个封装的组件定义的样式互相影响，最后导致样式出现错误调用致使页面混乱。

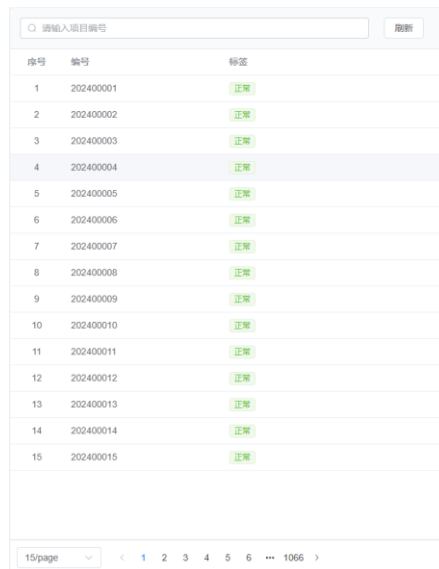
作为封装的组件，部分数据需要从父组件中获取，因此需要使用 `defineProps` 方法来定义变量列表，使用这个方法定义的变量会自动暴露给父组件，父组件中能够在页面结构中引用子组件的位置在标签中使用冒号+变量名字来向子组件传值。

同样，子组件也可以暴露方法给父组件，只需要在 `defineExpose` 方法中定义方法便可以自动暴露给父组件。但是父组件需要先在标签中添加`:ref="..."`来获取组件实例，然后才能使用组件实例来调用组件的方法。

5.4 数据分析

数据分析页面采用主从式布局设计，左侧为数据列表展示区，右侧为详情分析面板，通过智能交互设计实现高效的数据浏览与分析体验。

数据表格展示区域，如图 5.13 所示，采用 Element Plus 的 `el-table` 组件进行强化设计，支持多维度数据展示与智能交互。表格默认显示数据 ID、提交时间、投保人基本信息等核心字段，通过状态标签直观标识每条记录的欺诈风险评估结果。对于确认为欺诈的数据，表格会额外显示经过标准化的欺诈类型分类，如"虚假诊疗"、"超额收费"或"冒名就医"等专业标识。



序号	编号	状态
1	202400001	正常
2	202400002	正常
3	202400003	正常
4	202400004	正常
5	202400005	正常
6	202400006	正常
7	202400007	正常
8	202400008	正常
9	202400009	正常
10	202400010	正常
11	202400011	正常
12	202400012	正常
13	202400013	正常
14	202400014	正常
15	202400015	正常

图 5.13 数据表格

表格顶部配置了智能搜索栏，采用防抖技术优化查询体验。当用户输入数据编号时，系统会在输入停止 300 毫秒后自动触发搜索，避免频繁请求造成的性能压力。搜索功能支持模糊匹配与精确查询两种模式：输入完整 ID 时执行精确查找直接定位到特定记录；

输入部分字符时则显示匹配的所有结果列表。

点击任意数据行时，系统通过异步加载机制在右侧详情面板展示深度分析结果。面板顶部显示该条数据的风险等级仪表盘，采用动态可视化设计直观呈现欺诈概率数值。底部医疗费用分析模块采用玫瑰图对比显示门诊费、住院费、检查费、药品费的具体分布情况。数据分析详情部分页面如图 5.14 所示。



图 5.14 详细数据分析

由于涉及的数据量非常大，直接将后端数据放入表单中会导致严重卡顿，所以利用 ElementPlus 的 Pagination 组件对数据进行分组处理，每一次前端浏览器只需要渲染 15 条左右的数据，避免了过多数据渲染导致的卡断甚至崩溃，降低了前端浏览器的渲染负担，优化用户体验。

5.5 欺诈名单

欺诈名单页面采用了和数据分析页面相似的页面布局，降低用户对于不同功能页面的学习成本，优化用户体验。

欺诈名单页面单独筛选出所有的异常数据，用户能针对性地浏览异常数据，对所有的异常数据进行分析与处理。针对单条数据，用户可以自行判断该数据具体是否为欺诈数据，若认为该数据暂时无法断定为欺诈数据，则可以将该数据从数据列表中删除，删除弹窗如图 5.15 所示。



图 5.15 删除欺诈类型

若用户认为当前数据的欺诈类型系统分类有误，则可以根据自身理解，将数据的欺诈类型修改为用户自己认为的欺诈类型，修改弹窗如图 5.16 所示。这样的修改会持久化到数据库中，删除操作不会去掉数据库中的记录，而是将该条记录的是否异常字段修改为“否”，保留原始数据，方便后续对数据进行再次利用或检查。



图 5.16 修改欺诈数据

在右侧分析界面中，针对当前条目的异常分析，我们新增了关键的可视化工具：各类医疗消费的“申报金额与总花费金额散点图”。该图表以医疗总费用作为纵轴（Y 轴），以患者申报费用作为横轴（X 轴），直观展示两者之间的关系。其核心价值在于清晰呈现申报费用在总费用中的占比情况。每个散点代表一项具体消费条目，其位置直接反映

了申报金额占总费用的比例。这张图通过并置对比总费用与申报费用，为用户提供了一个高效识别潜在异常数据点的强大视觉分析工具。散点图如图 5.17 所示。

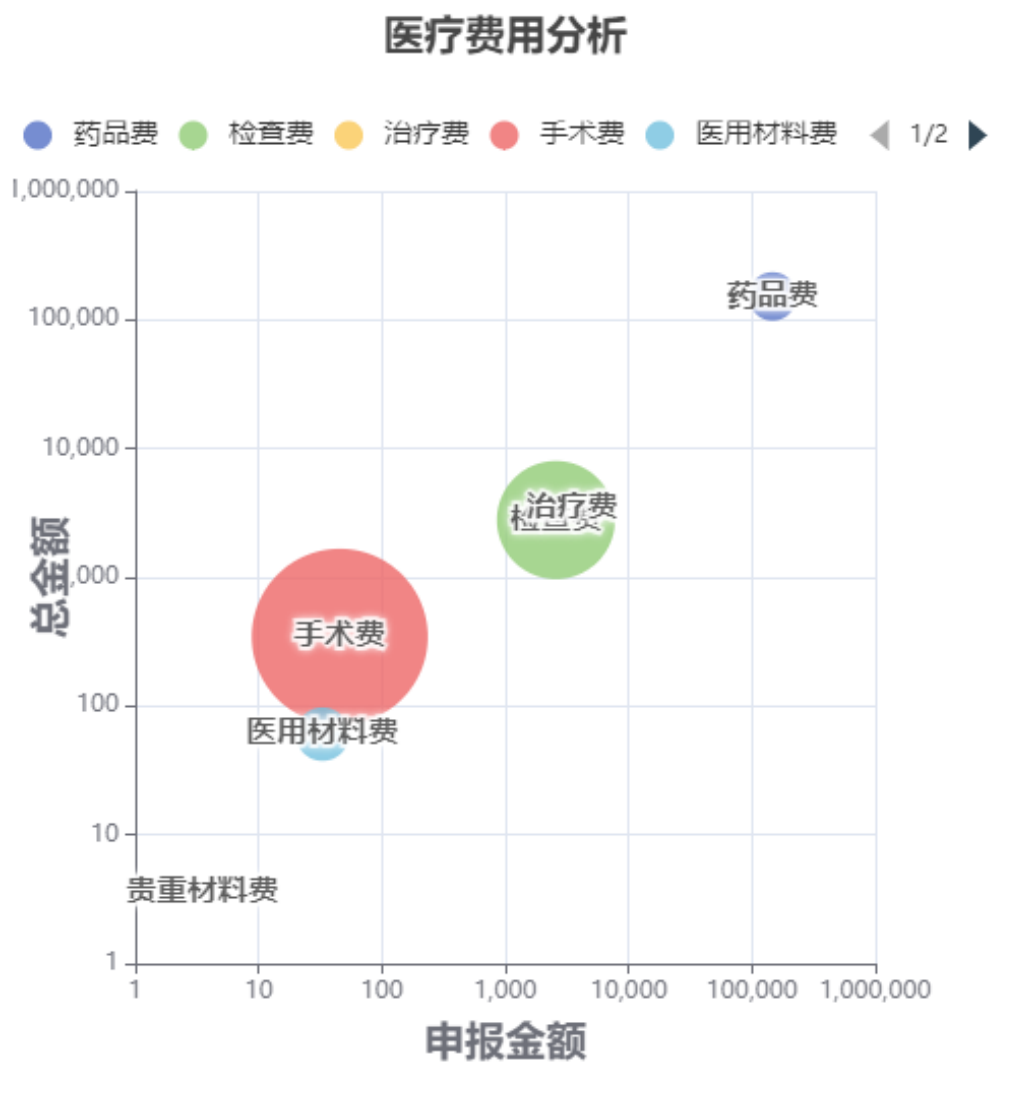
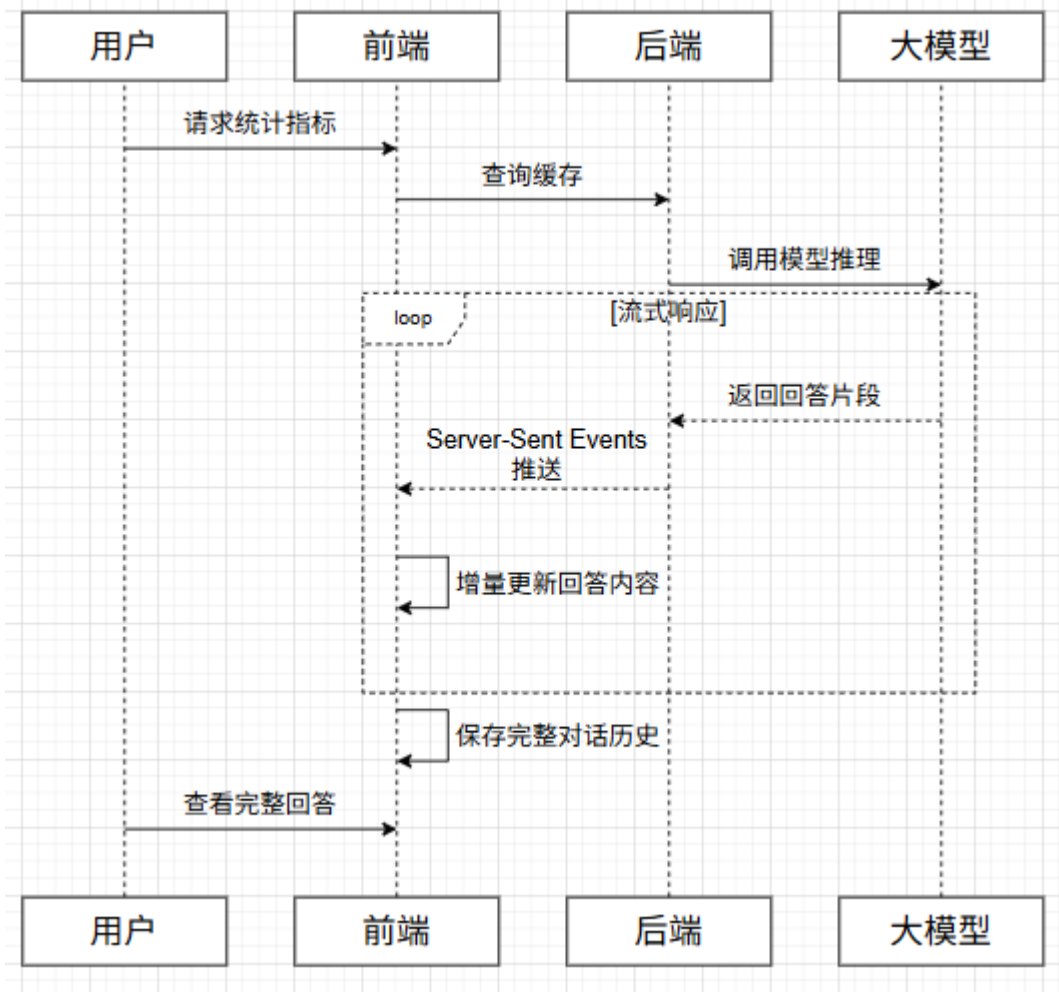


图 5.17 医疗费用散点图

5.6 医保欺诈问答

考虑到用户可能对于医保相关的法律法规了解可能不深，或是对医保相关知识尚有欠缺，因此在系统中接入了经过微调的大语言模型。大模型本身部署在本地，基于 deepseek-r1 模型进行微调，增加模型针对医疗保险相关的专业度，用户可以将问题发给大语言模型进行询问，大模型会给出浅显易懂的回答。大模型回答流程如图 5.18 所示。



5.18 问答模块响应时序图

由于是基于 deepseek 的模型进行微调，所以模型的回答数据并非是正常模型的一整段回答返回，而是以输出流的方式回答，所以需要对输出进行特殊处理，以适配前端的展示。

5.7 数据检测

数据检测模块接入了 Spark 的流处理模型，能够以数据流的形式来实时处理数据。用户能选择一个包含医保数据的 csv 文件，上传后前端会对文件进行解析，然后解析文件内容，将解析完成的数据一条条地传输给后端，后端将收到的数据通过 Kafka 生产者传入 Topic，然后 Spark 从 Kafka 消费者中获得数据进行分析，最后将分析结果返回给前端。

在文件上传功能的实现中，我们基于 ElementPlus 的 el-upload 组件构建了高效的交互体系。当用户通过拖拽区域或文件选择器提交文档时，前端界面立即触发视觉反馈机制：环形进度条动态展示传输百分比，状态图标实时标示当前上传阶段。为应对大

数据量场景下的性能挑战，系统采用异步处理架构解耦数据传输与计算过程——文件分块上传后，前端即刻返回"接收成功"状态提示，同时启动加载动画过渡效果，而底层则通过 Axios 的 Promise 封装将文件流异步传输至后端服务，确保主界面交互不被阻塞。

面对后端数据处理可能产生的延迟，系统建立三重保障机制：首先在文件接收完成后，后端返回包含任务 ID 的 202 Accepted 响应，前端随即启动 WebSocket 长连接持续监听处理进度，在界面展示动态更新的进度环。服务端将计算密集型任务加入 RabbitMQ 消息队列，由专用计算节点执行大数据分片处理与格式转换，特别是针对超过 100MB 的文件自动启用 MapReduce 并行处理框架。当最终处理完成信号通过 WebSocket 通道传回时，前端智能发起聚合请求，统一获取主数据、分析指标和过程日志，确保所有关联数据准备就绪后才进行整体渲染。文件上传如图 5.19 所示。

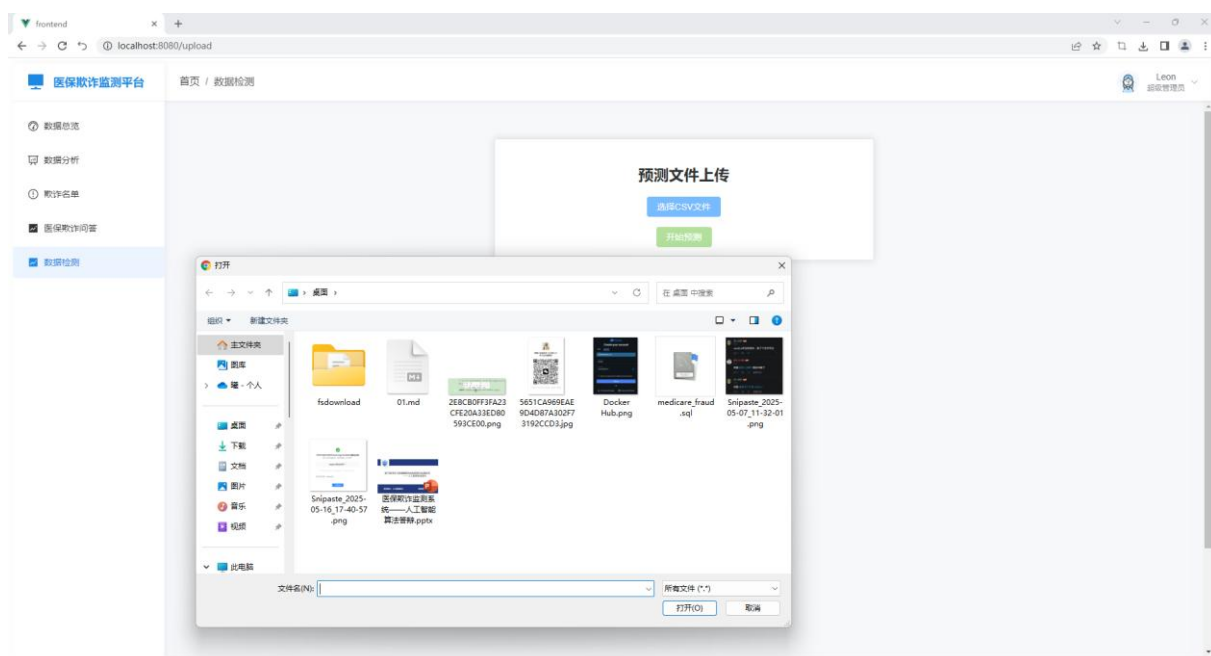


图 5.19 文件上传

6 总结与展望

6.1 总结

本篇报告围绕“基于联邦学习与模糊聚类的医保欺诈监测系统”的前后端及数据可视化模块，完成了从架构设计到代码实现的全流程开发。

在技术上，采用前后端分离架构，前端基于 Vue3 + TypeScript + ElementPlus 构建响应式管理界面，后端基于 FastAPI 实现高性能 RESTful API 服务，通过组合式 API 与 TypeScript 强类型校验提升代码健壮性，利用 ECharts 实现欺诈因子环形图、费用分布散点图等动态图表，支持千万级数据的异步加载与分页渲染。

在功能上，通过 Redis 的验证码生命周期管理，结合 SMTP 邮件服务和路由守卫机制，能有效保障系统隐私数据安全。项目基本实现了对于医保欺诈数据的监测和判断，通过 SQLAlchemy 异步查询实现实时数据统计，结合 ElementPlus 表格分页与动态过滤优化用户体验。

总而言之，本项目基本实现预定的开发任务，为医保欺诈防控提供了可落地的技术工具链，未来将通过算法迭代与架构升级，持续赋能医保基金监管的智能化转型。

6.2 设计成果亮点

- 1、通过模块化设计显著提升代码复用率，降低维护成本。
- 2、支持异常数据动态编辑与软删除，保留审计追踪能力。
- 3、验证了联邦学习与模糊聚类模型在实际业务中的可落地性，为医保部门提供欺诈识别率提升的技术路径。

6.3 项目局限性

本项目在开发过程中，存在许多尚未解决的问题。

首先是针对大规模数据的读写与渲染，目前的系统当出现需要后端读取大量数据像前端传递时，会出现显著的卡顿，虽然卡顿时间不算太长，但是会显著影响用户的使用体验。在后续的优化中，可以考虑前端采用分段式异步请求，而非一次性请求全部数据，并且采用异步加载的方法来降低前端渲染压力，而后端同样采用异步查询，同时增加代码的并发性，加快查询速度。

接着是实时检测部分。目前的版本并没有实现真正的流处理，前后端无法向 spark 集群传输流式数据，且 spark 集群端存在处理时间过长，响应时间过长的问題，后续优化中考虑接入实时数据产生源，比如和当地医院合作，实时获得每日患者的医院就诊记录，实现流处理。并且当前的实时监测使用的是新训练的轻量化 SparkMLlib 模型，而并非数据库中生成分析数据时使用的主模型，任需要单独运行模型来批量分析数据。在后续的开发中需要将主模型接入到流处理步骤中，替代掉 Spark 集群。

最后是针对的医保数据类型过于狭窄。目前的项目仅能针对结构化的医保信息进行分析和处理，但是在实际场景中更多的是非结构化文本的欺诈信息，后续需要集成针对非集成化信息处理的模块。



参考文献

- [1] FastAPI. FastAPI 官方文档[EB/OL]. (2023). <https://fastapi.tiangolo.com/zh/>.
- [2] Vue.js. Vue.js 官方文档（中文）[EB/OL]. (2024). <https://cn.vuejs.org/>.
- [3] Element Plus. Element Plus 官方文档（中文）[EB/OL]. (2024). <https://element-plus.org/zh-CN/>.
- [4] Apache ECharts. Apache ECharts 官方文档（中文）[EB/OL] (2024). <https://echarts.apache.org/zh/index.html>.
- [5] SQLAlchemy. SQLAlchemy 2.0 中文文档[EB/OL]. (2023)[2024-07-06]. <https://docs.sqlalchemy.org.cn/en/20/intro.html>.
- [6] 陈起风.“救命钱”沦为“唐僧肉”:内在逻辑与治理路径——基于百余起骗保案的实证研究[J].社会保障研究,2019,(04):42-51.
- [7] 项露芬,孙佳怡,李梦婷.基于 Vue 和 Node.js 的音乐门票管理系统的设计与实现[J].现代信息科技,2025,9(11):96-101.DOI:10.19850/j.cnki.2096-4706.2025.11.019.
- [8] 詹长春,索丽.大数据背景下医保欺诈基金监测研究[J].中国农村卫生事业管理,2025,45(04):292-296.DOI:10.19955/j.cnki.1005-5916.2025.04.011.
- [9] 张正卿,车惯红.以大数据和人工智能打造医保反欺诈模型[J].上海信息化,2025,(01):45-48.
- [10] 周群威.基于 ECharts 的多维度动态图表的异常分析和解决方法[J].电脑编程技巧与维护,2025,(05):151-154.DOI:10.16184/j.cnki.comprg.2025.05.031.
- [11] 麦冬,陈涛,梁宗湾.轻量级响应式框架 Vue.js 应用分析[J].信息与电脑(理论版),2017,(07):58-59.
- [12] 陈旭,许嘉杰,陈晓婷,等.基于 Typescript 的事件订阅-发布机制探究与实现[J].电脑编程技巧与维护,2022,(05):17-20.DOI:10.16184/j.cnki.comprg.2022.05.051.



附录 部分代码

数据总览页面组件构建实现

```
<template>
  <div class="dashboard-container">
    <PanelGroup
      :total="total"
      :normal="normal"
      :abnormal="abnormal"
      :abnormal-rate="abnormalRate"
    ></PanelGroup>

    <el-row :gutter="20">
      <el-col :span="12">
        <div class="chart-wrapper">
          <PieChart />
        </div>
      </el-col>
      <el-col :span="12">
        <div class="chart-wrapper">
          <BubbleView />
        </div>
      </el-col>
    </el-row>
  </div>
</template>
```

前端玫瑰图组件实现

```
<script setup lang="ts">
import { defineProps, onMounted } from 'vue';
import { ref } from 'vue';
import * as echarts from 'echarts';

onMounted(() => {
  initChart();
})

const props = defineProps({
  classname: {
    type: String,
    default: 'pie-chart'
  },
  height: {
```



```
        type: String,
        default: '550px'
      },
      width: {
        type: String,
        default: '100%'
      }
    }
  });

const pieChart = ref<HTMLElement | null>(null);
let chartInstance: echarts.ECharts | null = null;

const initChart = () => {
  if (pieChart.value) {
    chartInstance = echarts.init(pieChart.value);

    const option = {
      title: {
        text: '特征因子分布',
        left: '18%',
        top: 10,
        textStyle: {
          fontSize: 35,
          fontWeight: 'bold'
        }
      },
      tooltip: {
        trigger: 'item',
        position: function (point: any) {
          return [point[0] + 20, point[1] - 40];
        },
        formatter: '{a} <br/>{b}'
      },
      legend: {
        orient: 'vertical',
        right: 0,
        top: 10,
        data: [
          '基本统筹基金支付金额_SUM',
          '本次审批金额_SUM',
          '药品费申报金额_SUM RA',
          '药品费发生金额_SUM',
          '月药品金额_MAX',
          '月统筹金额_AVG',
          '治疗费申报金额_SUM',

```




```
],
formatter: function(name: any) {
    return name.length > 5 ? name.slice(0, 5) + '...' : name;
}
},
series: [
    {
        name: '特征因子',
        type: 'pie',
        radius: ['5%', '14%'],
        center: ['35%', '55%'],
        emphasis: {
            focus: 'self',
            blurScope: 'global'
        },
        label: {
            show: false,
        },
        labelLine: {
            show: false
        },
        data: [
            {
                value: 20,
                name: '治疗费申报金额_SUM',
                label: {
                    show: true,
                    position: 'outside', // 显示在外部
                    alignTo: 'edge',
                    margin: 8,
                    formatter: function(params: any) {
                        // 只显示前 5 字+省略号
                        return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
                    }
                },
                labelLine: { show: true }
            },
            {
                value: 80,
                name: '',
                itemStyle: { color: '#fff' },
                tooltip: { show: false },
                cursor: 'default'
            }
        ]
    },
],
```



```
labelLayout: function(params: any) {
  if (params.labelRect && params.dataIndex === 0) {
    return {
      y: params.labelRect.y - 40,
      x: params.labelRect.x + 100
    };
  }
},
{
  name: '特征因子',
  type: 'pie',
  radius: ['15%', '24%'],
  center: ['35%', '55%'],
  emphasis: {
    focus: 'self',
    blurScope: 'global'
  },
  label: {
    show: false
  },
  labelLine: {
    show: false
  },
  data: [
    {
      value: 30,
      name: '月统筹金额_AVG',
      label: {
        show: true,
        position: 'outside', // 显示在外部
        alignTo: 'edge',
        margin: 8,
        formatter: function(params: any) {
          // 只显示前 5 字+省略号
          return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
        }
      },
      labelLine: { show: true }
    },
    {
      value: 70,
      name: '',
      itemStyle: { color: '#fff' },
      tooltip: { show: false },
```



```

        cursor: 'default'
    }
],
labelLayout: function(params: any) {
    if (params.labelRect && params.dataIndex === 0) {
        return {
            y: params.labelRect.y - 20,
            x: params.labelRect.x + 100
        };
    }
},
{
    name: '特征因子',
    type: 'pie',
    radius: ['25%', '34%'],
    center: ['35%', '55%'],
    emphasis: {
        focus: 'self',
        blurScope: 'global'
    },
    label: {
        show: false,
    },
    labelLine: {
        show: false
    },
    data: [
        {
            value: 40,
            name: '月药品金额_MAX',
            label: {
                show: true,
                position: 'outside', // 显示在外部
                alignTo: 'edge',
                margin: 8,
                formatter: function(params: any) {
                    // 只显示前 5 字+省略号
                    return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
                }
            },
            labelLine: { show: true }
        },
        {
            value: 60,

```



```
        name: '',
        itemStyle: { color: '#fff' },
        tooltip: { show: false },
        cursor: 'default'
    }
],
labelLayout: function(params: any) {
    if (params.labelRect && params.dataIndex === 0) {
        return {
            x: params.labelRect.x + 100
        };
    }
}
},
{
    name: '特征因子',
    type: 'pie',
    radius: ['35%', '44%'],
    center: ['35%', '55%'],
    emphasis: {
        focus: 'self',
        blurScope: 'global'
    },
    label: {
        show: false,
    },
    labelLine: {
        show: false
    },
    data: [
        {
            value: 50,
            name: '药品费发生金额_SUM',
            label: {
                show: true,
                position: 'outside', // 显示在外部
                alignTo: 'edge',
                margin: 8,
                formatter: function(params: any) {
                    // 只显示前 5 字+省略号
                    return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
                }
            },
            labelLine: { show: true }
        },
    ],
}
```



```
{
  value: 50,
  name: '',
  itemStyle: { color: '#fff' },
  tooltip: { show: false },
  cursor: 'default'
}
],
labelLayout: function(params: any) {
  if (params.labelRect && params.dataIndex === 0) {
    return {
      x: params.labelRect.x + 100
    };
  }
}
},
{
  name: '特征因子',
  type: 'pie',
  radius: ['45%', '54%'],
  center: ['35%', '55%'],
  emphasis: {
    focus: 'self',
    blurScope: 'global'
  },
},
label: {
  show: false,
},
labelLine: {
  show: false
},
data: [
  {
    value: 60,
    name: '药品费申报金额_SUM RA',
    label: {
      show: true,
      position: 'outside', // 显示在外部
      alignTo: 'edge',
      margin: 8,
      formatter: function(params: any) {
        // 只显示前 5 字+省略号
        return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
      }
    }
  },
},
```



```

        labelLine: { show: true }
    },
    {
        value: 40,
        name: "",
        itemStyle: { color: '#fff' },
        tooltip: { show: false },
        cursor: 'default'
    }
],
labelLayout: function(params: any) {
    if (params.labelRect && params.dataIndex === 0) {
        return {
            x: params.labelRect.x + 100
        };
    }
}
},
{
    name: '特征因子',
    type: 'pie',
    radius: ['55%', '64%'],
    center: ['35%', '55%'],
    emphasis: {
        focus: 'self',
        blurScope: 'global'
    },
    label: {
        show: false,
    },
    labelLine: {
        show: false
    },
    data: [
        {
            value: 70,
            name: '本次审批金额_SUM',
            label: {
                show: true,
                position: 'outside', // 显示在外部
                alignTo: 'edge',
                margin: 8,
                formatter: function(params: any) {
                    // 只显示前 5 字+省略号
                    return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
                }
            }
        }
    ]
}

```



```
    }
  },
  labelLine: { show: true }
},
{
  value: 30,
  name: "",
  itemStyle: { color: '#fff' },
  tooltip: { show: false },
  cursor: 'default'
}
],
labelLayout: function(params: any) {
  if (params.labelRect && params.dataIndex === 0) {
    return {
      x: params.labelRect.x + 100
    };
  }
}
},
{
  name: '特征因子',
  type: 'pie',
  radius: ['65%', '74%'],
  center: ['35%', '55%'],
  emphasis: {
    focus: 'self',
    blurScope: 'global'
  },
},
label: {
  show: false,
},
labelLine: {
  show: false
},
data: [
  {
    value: 80,
    name: '基本统筹基金支付金额_SUM',
    label: {
      show: true,
      position: 'outside', // 显示在外部
      alignTo: 'edge',
      margin: 8,
      formatter: function(params: any) {
```




```

// 只显示前 5 字+省略号
return params.name.length > 5 ? params.name.slice(0, 5) + '...' : params.name;
    }
  },
  labelLine: { show: true }
},
{
  value: 20,
  name: "",
  itemStyle: { color: '#fff' },
  tooltip: { show: false },
  cursor: 'default'
}
],
labelLayout: function(params: any) {
  if (params.labelRect && params.dataIndex === 0) {
    return {
      x: params.labelRect.x + 100
    };
  }
}
},
]
};

chartInstance.setOption(option);
}
}
</script>

```

用户登录验证码逻辑实现

```
async def send_verification_code(self, email: str, scene: str = "login"):
```

```
    # 检查邮箱格式
```

```
    if not email or "@" not in email:
```

```
        return False, "无效的邮箱格式"
```

```
    async with managed_async_session() as db:
```

```
        try:
```

```
            # 根据场景进行额外检查
```

```
            if scene == "register":
```

```
                # 检查邮箱是否已注册
```

```
                result = await db.execute(select(UserInfo).where(email == UserInfo.username))
```

```
                user = result.scalars().first()
```

```
                if user:
```

```
                    return False, "该邮箱已被注册"
```

```
            # 获取 Redis 连接
```



```

        r = await RedisConnector.get_connection()
        # 频率限制检查（同一邮箱 1 分钟内只能发送一次）
        if await r.exists(f'rate_limit:{email}'):
            return False, "请求过于频繁，请稍后再试"
        # 生成 6 位随机验证码
        code = ''.join(random.choices('0123456789', k=6))
        # 存储到 Redis (5 分钟有效期)，key 中包含场景信息
        await r.setex(f'verification_code:{scene}:{email}', 300, code)
        # 设置频率限制标记（1 分钟内不能再次发送）
        await r.setex(f'rate_limit:{email}', 60, "1")
        # 发送邮件
        result_sent = await self.email_service.send_verification_email(email, code)
        return result_sent, "验证码已发送" if result_sent else "邮件发送失败"
    except Exception as e:
        logger.error(f"发送验证码时出错: {str(e)}")
        return False, "服务器内部错误"

# Redis 数据库连接池建立
def init_pool(cls):
    # 建立连接池
    try:
        if cls._pool is None:
            REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379")
            logger.info(f"创建 Redis 连接池: {REDIS_URL}")
            # 使用 from_url 方法创建连接池
            cls._pool = redis.ConnectionPool.from_url(
                REDIS_URL,
                max_connections=10,
                decode_responses=True
            )
        return cls._pool
    except Exception as e:
        logger.error(f"初始化 Redis 连接池失败: {str(e)}")
        raise

@classmethod
async def get_connection(cls):
    # 处理 Redis 的数据操作
    try:
        # 懒加载机制，处理 Redis 数据库数据操作前，才建立连接池
        if not cls._pool:
            cls.init_pool()
        return redis.Redis(connection_pool=cls._pool)
    except Exception as e:
        logger.error(f"获取 Redis 连接失败: {str(e)}")
        raise

```